MODPKICA Java 复刻版

开发助手 • 源码分析

版本：v1.28-r3（2026-10-02）

基于最新源码生成 · BC 1.85 离线全家桶

涵盖：X.509 证书体系 · CRL · OCSP · 时间戳 · RFC 6962 CT 日志 · 微软代码签名 · 国密 PDF 签名 · Web 管理面板

作者：@魔帝本尊 QQ：2829969554

本系统仅供学习 x.509 ASN.1 编码使用，作者不承担任何由此产生的一切问题。

目录

第一章 项目总览 5

1.1 源码构成（23 个类） . . . 5

1.2 技术栈与运行环境 . . . 5

1.3 构建与运行 . . . 5

1.4 PKI 目录结构 . . . 6

第二章 HTTP 监听链接（网络服务全路由详解） 7

2.1 路由总表 . . . 7

2.2 OCSP 接口 /OCSP 与 /ocsp . . . 7

2.3 CRL / CRT 静态发布 . . . 7

2.4 时间戳接口（Authenticode 与 RFC 3161） . . . 7

2.5 CT 日志服务 /ct/add（RFC 6962） . . . 8

2.6 CT 标准 API /ct/v1/*（RFC 6962 §4，v1.28-r3 新增） . . . 8

POST /ct/v1/add-chain . . . 8

POST /ct/v1/add-pre-chain . . . 8

GET /ct/v1/get-sth . . . 8

GET /ct/v1/get-roots . . . 8

GET /ct/v1/get-entries . . . 9

GET /ct/v1/get-proof-by-hash . . . 9

GET /ct/v1/get-sth-consistency . . . 9

GET /ct/v1/get-entry-and-proof . . . 9

2.7 Web 管理面板（8080） . . . 9

第三章 主程序详解（7 个） 10

MAIN —— MAIN.java . . . 10

常量 . . . 10

方法一览（41 个） . . . 10

Admin —— Admin.java . . . 11

常量 . . . 11

方法一览（22 个） . . . 11

MakeRoot —— MakeRoot.java . . . 12

常量 . . . 12

方法一览（15 个） . . . 12

MakeCert —— MakeCert.java . . . 13

常量 . . . 13

方法一览（40 个） . . . 13

RootGetCrl —— RootGetCrl.java . . . 15

常量 . . . 15

方法一览（9 个） . . . 15
CertVerify —— CertVerify.java . . . 16
常量 . . . 16
方法一览（62 个） . . . 16
Auto —— Auto.java . . . 18
常量 . . . 18
方法一览（10 个） . . . 18
各主程序运行示例 . . . 19
第四章 工具类详解（16 个） 20
AppEnv —— AppEnv.java . . . 20
方法一览（1 个） . . . 20
AppPaths —— AppPaths.java . . . 20
方法一览（1 个） . . . 20
Bc —— Bc.java . . . 20
方法一览（1 个） . . . 20
CertUtil —— CertUtil.java . . . 20
常量 . . . 20
方法一览（18 个） . . . 20
CertsDB —— CertsDB.java . . . 21
常量 . . . 21
方法一览（15 个） . . . 22
Config —— Config.java . . . 22
常量 . . . 22
方法一览（10 个） . . . 22
CrlUtil —— CrlUtil.java . . . 23
方法一览（3 个） . . . 23
CtLog —— CtLog.java . . . 23
常量 . . . 23
方法一览（37 个） . . . 23
MLog —— MLog.java . . . 25
常量 . . . 25
方法一览（5 个） . . . 25
OcspUtil —— OcspUtil.java . . . 25
常量 . . . 25
方法一览（7 个） . . . 25
PemIO —— PemIO.java . . . 26
方法一览（12 个） . . . 26
SignTool —— SignTool.java . . . 27
常量 . . . 27

方法一览（69 个） . . . 27
Sm2Util —— Sm2Util.java . . . 31
常量 . . . 31
方法一览（11 个） . . . 31
TimestampUtil —— TimestampUtil.java . . . 31
常量 . . . 31
方法一览（8 个） . . . 32
WebAdmin —— WebAdmin.java . . . 32
常量 . . . 32
方法一览（114 个） . . . 33
X509JsonConverter —— X509JsonConverter.java . . . 37
常量 . . . 37
方法一览（20 个） . . . 37
第五章 附录 **39**
5.1 构建脚本用法（build.cmd / build.sh） . . . 39
5.2 全局配置参数（PKI/CONFIG.txt） . . . 39
5.3 日志格式（PKI/logs.log） . . . 39

第一章 项目总览

MODPKICA Java 复刻版是把原有 Go 版 modpkica 用 Java + BouncyCastle 1.85 完整重写的 PKI（公钥基础设施）离线工具链，包含 7 个主程序与 16 个工具类，覆盖 X.509 证书签发/解析/验证、CRL 吊销列表、OCSP 在线状态协议、RFC 3161 时间戳、RFC 6962 证书透明度（CT）日志、微软 Authenticode 代码签名、Adobe/国密 PDF 文档签名，以及内置 Web 管理面板（监听 8080）与网络服务端（监听 80）。

1.1 源码构成（23 个类）

源文件	类	KB	方法数	常量数
Admin.java	Admin	17	22	2
AppEnv.java	AppEnv	0	1	0
AppPaths.java	AppPaths	1	1	0
Auto.java	Auto	9	10	2
Bc.java	Bc	0	1	0
CertUtil.java	CertUtil	26	18	3
CertVerify.java	CertVerify	77	62	2
CertsDB.java	CertsDB	11	15	4
Config.java	Config	4	10	1
CrlUtil.java	CrlUtil	4	3	0
CtLog.java	CtLog	51	37	3
MAIN.java	MAIN	55	41	3
MLog.java	MLog	3	5	1
MakeCert.java	MakeCert	50	40	2
MakeRoot.java	MakeRoot	23	15	4
OcspUtil.java	OcspUtil	16	7	5
PemIO.java	PemIO	14	12	0
RootGetCrl.java	RootGetCrl	14	9	1
SignTool.java	SignTool	131	69	26
Sm2Util.java	Sm2Util	8	11	5
TimestampUtil.java	TimestampUtil	10	8	3
WebAdmin.java	WebAdmin	408	114	15
X509JsonConverter.java	X509JsonConverter	107	20	14

1.2 技术栈与运行环境

· JDK：Java 26（兼容 JDK 11+，未使用高版本专有 API）
· 密码学库：BouncyCastle 1.85 离线全家桶（BC185/ 目录，bcprov/bcpkix/bcutil/bctls/bcpg 等，含国密 SM2/SM3 与 PQC ML-DSA）
· HTTP 服务：JDK 内置 com.sun.net.httpserver，无外部 Servlet 容器
· 前端：WebAdmin 内置单文件 HTML/JS（无框架、无 CDN 依赖，完全离线可用）
· 数据库：无——全部为明文文件（CERTS.txt / myCERTS.txt / CONFIG.txt / logs.log / CT 日志 JSON）

1.3 构建与运行

```
build.cmd 1    # 全量编译（javac -Xlint:all -encoding UTF-8，编译成功零警告）
build.cmd 2    # 编译后自动运行 MAIN（80 端口网络服务 + 8080 Web 管理）
build.cmd 9    # 单元测试（162 passed / 0 failed）
# 直接运行示例：
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.MAIN
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.Admin initOCSP
java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.MakeRoot "CN=Root,O=Org,C=CN" 2048 sha256 RSA
```

1.4 PKI 目录结构

```
PKI/
├── CERTS.txt            证书登记簿（# 序列号 类型R/C/E 状态V/R 算法 哈希 生效 失效 颁发者序列号）
├── myCERTS.txt          我的证书库（pfx/jks 导入，模板 myCERTS.txt.sm）
├── CONFIG.txt           系统配置（WEBPORT/OCSP/CRL/CRT/CPS/CPS_Nonice/suport）
├── logs.log             操作日志（追加式）
├── ROOT/  CA/  CERT/  KEY/  OCSP/  TIMSTAMP/    证书/私钥分区
├── WebPublic/CRT/  WebPublic/CRL/               HTTP 对外发布区
├── CTLOG/               modpkica_ct_list.json（CT 日志密钥库）、ctlogs.txt（CT 请求日志）、trees/（Merkle 树）
└── MY/                  pfx/jks 导入的证书与私钥
```

第二章 HTTP 监听链接（网络服务全路由详解）

MAIN 服务端监听端口默认 80（CONFIG.txt 的 WEBPORT 参数），Web 管理面板监听 8080（CONFIG.txt 的 suport 参数，默认 8080，文件不存在或未配置时自动创建/新增）。以下为 MAIN 服务全部监听路由，均以 com.sun.net.httpserver 注册。

2.1 路由总表

路由	方法	Handler	说明
/OCSP	POST	handleOcsp	OCSP 在线证书状态协议查询接口（RSA/ECC/SM2，摘要自适应）
/ocsp	POST	handleOcsp	OCSP 小写别名，与 /OCSP 等价
/timestamp/sha1	POST	handleTimestampSha1	微软 Authenticode 时间戳（SHA1）
/timestamp	POST	handleTimestampSha256	微软 Authenticode 时间戳（SHA256，SM2/SM3 自适应）
/rfc3161	POST	handleRfc3161	RFC 3161 文档签名专用时间戳（SHA1/SHA256 自适应）
/ct/add	POST	handleCtAdd	CT 日志服务（RFC 6962）：接收 PEM 预证书 + 颁发者证书，签发 SCT 列表
/ct/v1/*	GET/POST	handleCtV1	CT 标准 API（RFC 6962 §4）：8 个子接口
/crl	GET	handleFile	CRL 静态发布：/CRL/{序列号}.crl（见下）
/crt	GET	handleFile	CRT 静态发布：/CRT/{序列号}.crt
/	GET	handleFile	WebPublic 静态文件服务（网页/证书/CRL 对外目录）

2.2 OCSP 接口 /OCSP 与 /ocsp

接收 ASN.1 DER 编码的 OCSPRequest（application/ocsp-request），返回 DER 编码 OCSPResponse（application/ocsp-response）。支持 RSA / ECC / SM2 三类证书与密钥对，签名摘要算法自适应：SM2 固定 SM3，RSA/ECC 按请求 CertID 中携带的哈希算法（sha1/sha256/sha384/sha512）选择；请求未携带摘要方式时默认 SHA1。

· 目标证书序列号 → 从 CERTS.txt 定位记录 → 经颁发者序列号从 OCSP 目录加载 {颁发者序列号}_ocsp.crt/_ocsp.key 签名
· 未知序列号 → 返回无签名响应（不签名）；颁发者哈希不匹配同样返回无签名响应
· 证书已吊销 → 返回 revoked 状态（含吊销时间与原因码）；正常 → good；未知 → unknown
· 日志：每次请求写 logs.log（时间/时间戳/操作/完整命令/结果/错误详情）

```
# 请求示例（openssl ocsp 客户端）：
openssl ocsp -issuer PKI\CA\xxx.crt -cert PKI\CERT\yyy.crt \
  -url http://localhost:80/ocsp -CAfile PKI\ROOT\zzz.crt
```

2.3 CRL / CRT 静态发布

通过通用 Handler handleFile 提供 WebPublic 目录静态服务：

· /CRT/{序列号}.crt —— 证书发布（WebPublic\CRT 目录，所有类型证书均同步写入）
· /CRL/{序列号}.crl 与 /CRL/{序列号}.der.crl —— CRL 吊销列表发布（WebPublic\CRL 目录）
· / —— WebPublic 根目录静态文件（可放网页）
· 请求路径中的 /crl、/crt 小写会自动归一化为 /CRL、/CRT 再映射目录，路径穿越字符被拒绝

2.4 时间戳接口（Authenticode 与 RFC 3161）

· /timestamp/sha1 —— 微软 Authenticode 时间戳，SHA1 摘要，加载 TIMSTAMP/rsa_sha1.crt 或 ecc_sha1.crt
· /timestamp —— 微软 Authenticode 时间戳，SHA256（可自适应 SM2/SM3，加载 rsa_sha256 /

ecc_sha256 / sm2_sm3）
· /rfc3161 —— RFC 3161 文档签名专用时间戳，SHA1 与 SHA256 自适应
· 时间戳证书目录命名规则：{类型}_{哈希算法}.crt（rsa_sha1 / rsa_sha256 / ecc_sha1 / ecc_sha256 / sm2_sm3 / pqc_sha256），由 Admin initTIMESTAMP 按 CERTS.txt 中全部 R 类型根证书逐一生成

```
# RFC 3161 时间戳请求（tsq/tsr 为 openssl 工具）：
openssl ts -query -data file.bin -sha256 -cert -out req.tsq
curl -s -H "Content-Type: application/timestamp-query" --data-binary @req.tsq \
  http://localhost:80/rfc3161 -o resp.tsr
openssl ts -reply -in resp.tsr -text
```

2.5 CT 日志服务 /ct/add（RFC 6962）

接收 POST 数据，格式为 PEM 格式预证书 + PEM 格式中间证书（可含根证书）。流程：解析 PEM → 校验预证书含 poison 扩展（1.3.6.1.4.1.11129.2.4.3）且颁发者匹配 → 去掉 poison 扩展得到 TBS → 按 RFC 6962 §3.2 构造 precert_entry 待签名数据 → 从 PKI/CTLOG/modpkica_ct_list.json 随机挑选≤3 组状态可用且类型匹配（SM2 预证书选 SM2 密钥，RSA/ECDSA 选 RSA/ECDSA）的日志密钥对签名 → 返回 TLS 编码 SCTList。每次请求追加一行日志到 PKI/CTLOG/ctlogs.txt。

```
# 请求（curl）：
curl -s -X POST -H "Content-Type: application/x-pem-file" \
  --data-binary @precert_chain.pem http://localhost:80/ct/add
```

2.6 CT 标准 API /ct/v1/*（RFC 6962 §4，v1.28-r3 新增）

统一由 handleCtV1 分发，8 个子接口：add-chain / add-pre-chain / get-sth / get-roots / get-entries / get-proof-by-hash / get-sth-consistency / get-entry-and-proof。未知路径返回 404，参数校验错误返回 400，内部异常返回 500。Merkle 树持久化于 PKI/CTLOG/trees/{log_id hex}.json（JSON Lines 逐行追加、append-only）。

POST /ct/v1/add-chain

方法：POST
说明：接收 base64(DER) 证书链数组 chain[]（叶子+颁发者+根）。拒绝含 poison 扩展的正式证书。逐级校验 issuer/subject 匹配与签名后签发 SCT 并入树（x509_entry）。响应：{sct_version,id,timestamp,extensions,signature}

```
curl -s -X POST -H "Content-Type: application/json" --data-binary @body.json http://127.0.0.1:80/ct/v1/add-chain
```

POST /ct/v1/add-pre-chain

方法：POST
说明：接收 base64(DER) 预证书链，必须含 poison 扩展且≥2 个证书。issuerKeyHash = SHA-256(chain[1] 公钥 SPKI)，按 §3.2 precert_entry 构造签名输入并入树（precert_entry）。

```
curl -s -X POST -H "Content-Type: application/json" --data-binary @body.json http://127.0.0.1:80/ct/v1/add-pre-chain
```

GET /ct/v1/get-sth

方法：GET
说明：返回最新签名树头：{tree_size, timestamp, sha256_root_hash, tree_head_signature, log_id}。timestamp 取树最后一条目时间，tree_head_signature 为 version=0+sig_type=1+u64 ts+32B root 的日志密钥签名。

```
curl -s http://127.0.0.1:80/ct/v1/get-sth
```

GET /ct/v1/get-roots

方法：GET
说明：返回全部 R 类型根的 DER 证书 base64 数组（§4.7 要求 DER，非 PEM）。

```
curl -s http://127.0.0.1:80/ct/v1/get-roots
```

GET /ct/v1/get-entries

方法：GET
说明：参数 start、end（可带 log_id）。返回 {entries:[{leaf_input,extra_data}]}，越界返回 400。leaf_input 为 TLS 编码 MerkleTreeLeaf（version|leaf_type|timestamp|signed_entry|extensions）。

```
curl -s "http://127.0.0.1:80/ct/v1/get-entries?start=0&end=1"
```

GET /ct/v1/get-proof-by-hash

方法：GET
说明：参数 hash（base64 叶子哈希 = SHA-256(0x00||leaf_input)）、tree_size。返回 {leaf_index, audit_path[]}（§3.4.1/3.4.2 审计路径）。

```
curl -s "http://127.0.0.1:80/ct/v1/get-proof-by-hash?hash=xxx&tree_size=2"
```

GET /ct/v1/get-sth-consistency

方法：GET
说明：参数 first、second（first≤second）。返回 {consistency[]} 一致性证明；first=0 返回空数组（Google CT 行为）。

```
curl -s "http://127.0.0.1:80/ct/v1/get-sth-consistency?first=1&second=2"
```

GET /ct/v1/get-entry-and-proof

方法：GET
说明：参数 leaf_index、tree_size。返回 {leaf_input, extra_data, audit_path}。

```
curl -s "http://127.0.0.1:80/ct/v1/get-entry-and-proof?leaf_index=0&tree_size=2"
```

2.7 Web 管理面板（8080）

WebAdmin 内置单文件前端，主分类：仪表盘 / 签发根证书 / 签发证书 / 初始化环境 / 证书库（CERTS.txt + cacerts + 我的证书）/ 证书验证 / 系统配置 / 日志 / SCT 服务管理 / 模板管理 / 签名专区（微软代码签名 + PDF 文档签名）/ 调试专区。后端 API 全部为 /api/*（33 个以上），返回 JSON；写操作串行化并写 logs.log。

第三章 主程序详解（7 个）

MAIN —— MAIN.java

常量

常量	值	说明
QUIET	Boolean.getBoolean("modpkica.quiet")	安静模式开关：-Dmodpkica.quiet=true 时关闭请求级控制台日志（错误仍输出 stderr）。
SHA1_OID	"1.3.14.3.2.26"	CertID 哈希 OID：SHA1
MAX_BODY_BYTES	8L * 1024 * 1024	最大请求体：8MB（OCSP/时间戳/CT 请求均远小于此，仅防内存 DoS）。

方法一览（41 个）

方法	签名	说明
main	public static void main(String[] args) throws Exception	安静模式开关：-Dmodpkica.quiet=true 时关闭请求级控制台日志（错误仍输出 stderr）。
handleOcsp	private static void handleOcsp(HttpExchange ex) throws IOException	/OCSP 与 /ocsp 共用的处理器（Go 版两个 handler 逻辑相同）。
isKnownHash	private static boolean isKnownHash(String oid)	判断是否已知哈希 OID。
ocspHashName	private static String ocspHashName(String oid)	OCSP 请求哈希 OID → 签名哈希名（默认 sha1）。
handleFile	private static void handleFile(HttpExchange ex) throws IOException	/ 根路径：静态文件分发，文件根目录为 PKI/WebPublic。 路径规则与 Go 版一致：URL 中的 crl→CRL、crt→CRT 大小写归一化。
handleTimestampSha1	private static void handleTimestampSha1(HttpExchange ex) throws IOException	微软 Authenticode 时间戳 ========== ========== ==========
handleTimestampSha256	private static void handleTimestampSha256(HttpExchange ex) throws IOException	微软 Authenticode 时间戳 ========== ========== ==========
handleAuthenticode	private static void handleAuthenticode(HttpExchange ex, int digitype, String tsaGroup) throws IOException	Authenticode 时间戳处理器。 @param digitype 1=sha1 2=sha256 3=sha384 4=sha512 @param tsaGroup 自适应根选择组（"sha1"/"sha256"）
head	private static byte[] head(byte[] a, int max)	取字节数组前 max 个字节。
jcaHashName	private static String jcaHashName(String hashName)	RFC3161 时间戳签名哈希名 → JCA 摘要算法名。
extractAuthenticodeContent	private static byte[] extractAuthenticodeContent(byte[] data)	提取 Authenticode 被签名内容：找到 0x04 0x81/82/83 标记， 取其后的数据（长度头 1/2/3 字节，对齐 Go 版 bytes.Split 逻辑）。
handleRfc3161	private static void handleRfc3161(HttpExchange ex) throws IOException	RFC3161 时间戳 ========== ========== ==========
handleCtAdd	private static void handleCtAdd(HttpExchange ex) throws IOException	/ct/add：仅接受 POST。请求体 = PEM 预证书（含 poison 扩展）+ PEM 颁发者（中间）证书。 按 RFC 6962 §3.1/§3.2 构造 precert_entry 签名输入，用 PKI/CTLOG/modpkica_ct_list.json 中全部 usable 日志密钥 ECDSA-P256 签名，返回 SCTList（TLS 编码，二进制）， 可直接作为证书 1.3.6.1.4.1.11129.2.4
handleCtV1	private static void handleCtV1(HttpExchange ex) throws IOException	/ct/v1 统一分发入口。
handleCtAddChain	private static void handleCtAddChain(HttpExchange ex) throws Exception	POST /ct/v1/add-chain：提交正式证书链，返回该日志的 SCT（RFC 6962 §4.1）。
handleCtAddPreChain	private static void handleCtAddPreChain(HttpExchange ex) throws Exception	POST /ct/v1/add-pre-chain：提交预证书链，返回该日志的 SCT（RFC 6962 §4.2）。
handleCtGetSth	private static void handleCtGetSth(HttpExchange ex) throws Exception	GET /ct/v1/get-sth：最新签名树头（RFC 6962 §4.3）。可选 ?log_id= 指定日志，默认第一个 usable。
handleCtGetRoots	private static void handleCtGetRoots(HttpExchange ex) throws Exception	GET /ct/v1/get-roots：日志接受的根证书（RFC 6962 §4.7，来自 CERTS.txt type=R）。
handleCtGetEntries	private static void handleCtGetEntries(HttpExchange ex) throws Exception	GET /ct/v1/get-entries：批量取日志条目（RFC 6962 §4.6）。可选 ?log_id=。

方法	签名	说明
handleCtGetProofByHash	private static void handleCtGetProofByHash(HttpExchange ex) throws Exception	GET /ct/v1/get-proof-by-hash：按叶子哈希取审计证明（RFC 6962 §4.5）。可选 ?log_id=。
handleCtGetConsistency	private static void handleCtGetConsistency(HttpExchange ex) throws Exception	GET /ct/v1/get-sth-consistency：两棵树头间一致性证明（RFC 6962 §4.4）。可选 ?log_id=。
handleCtGetEntryAndProof	private static void handleCtGetEntryAndProof(HttpExchange ex) throws Exception	GET /ct/v1/get-entry-and-proof：按索引取条目 + 审计证明（RFC 6962 §4.8）。可选 ?log_id=。
parseB64Chain	private static java.util.List<byte[]> parseB64Chain(Object chainObj) throws IOException	解析 add-chain/add-pre-chain 的 chain 数组（base64 DER 证书）。
derToCerts	private static List<X509Certificate> derToCerts(java.util.List<byte[]> ders) throws Exception	DER 字节列表 → X509Certificate 列表。
verifyChainLinks	private static void verifyChainLinks(List<X509Certificate> chain) throws IOException	校验链的颁发者-使用者关系与逐级签名（RFC 6962 §4.1 链校验）。
resolveLogKey	private static CtLog.LogKey resolveLogKey(String baseDir, String logIdParam, boolean sm2Only) throws IOException	解析日志密钥：优先 ?log_id= 精确匹配；否则选第一个 usable 且类型匹配（sm2Only 时只选 SM2）的密钥。无可用密钥抛异常。
queryParam	private static String queryParam(HttpExchange ex, String name)	读取 GET query 参数（无则返回空串）。
intParam	private static int intParam(HttpExchange ex, String name, int def)	读取 GET 整数参数，缺失/非法返回默认值。
sendJson	private static void sendJson(HttpExchange ex, int code, String json) throws IOException	发送 JSON 响应。
jsonEscape	private static String jsonEscape(String s)	JSON 字符串字段转义（用于错误消息）。
pickTSACert	private static String[] pickTSACert(String hash)	自适应根选择 TSA 证书/私钥路径（对齐 Go 版 pickTSACert）。 sha1→rsa_sha1/ecc_sha1 随机；sha256→rsa_sha256/ecc_sha256 随机；sm3→sm2_sm3 固定。 随机选中后若文件缺失则回退另一组。
fileExists	private static boolean fileExists(String relPath)	都没有 → 回退 rsa（后续读取会给出明确报错）
findExisting	private static String findExisting(String... relPaths)	按顺序找第一个存在的文件路径（相对 baseDir）。
readAll	private static byte[] readAll(InputStream in) throws IOException	读取请求体全部字节（带上限，超限抛 IOException 拒绝）。
sendBytes	private static void sendBytes(HttpExchange ex, byte[] data, String contentType) throws IOException	发送字节响应。
sendText	private static void sendText(HttpExchange ex, int code, String text) throws IOException	发送文本响应。
writeFile	private static synchronized void writeFile(String relPath, byte[] data) throws IOException	写文件（相对 baseDir，自动创建父目录）。synchronized：多线程下文件写串行化， 避免 OCSP/时间戳/CT 的调试文件并发交错覆盖（Files.write 非原子）。
sha1Hex	private static String sha1Hex(byte[] data) throws Exception	SHA-1 十六进制。
toHex	private static String toHex(byte[] data)	字节 → 小写十六进制。
lastIndexOf	private static int lastIndexOf(byte[] data, byte[] needle)	在 data 中找子串最后一次出现的位置。
log	private static void log(String msg)	控制台请求日志（前缀时间；安静模式下静默，日志仍完整写入 PKI/logs.log）。

Admin —— Admin.java

常量

常量	值	说明
VERSION	"MODPKICA Java 复刻版 v1.28 (BC 1.85 离线)"	版本信息。
STDOUT_LOCK	new Object()	捕获被调程序输出到 stdout 的内容，返回字符串。 P2-05：System.setOut 是全局替换，多个请求并发 capture 会互相污染 stdout， 故用进程内互斥锁串行化 capture 段（可重入，同一线程嵌套调用安全）。

方法一览（22 个）

方法	签名	说明
main	public static void main(String[] args) throws Exception	版本信息。
printHelp	private static void printHelp()	—

方法	签名	说明
cmdInit	private static void cmdInit(Scanner sc, String baseDir) throws Exception	init：备份 + 模板 + 根 + OCSP + TSA。
cmdMakeRoot	private static void cmdMakeRoot(Scanner sc, String baseDir) throws Exception	init：备份 + 模板 + 根 + OCSP + TSA。
cmdSignCert	private static void cmdSignCert(Scanner sc, String baseDir) throws Exception	signCERT：签发证书（普通模式，参数对齐 Go 版 MODML[1..12]）。
cmdRevoke	private static void cmdRevoke(Scanner sc, String baseDir) throws Exception	RevokeCERT：吊销证书并刷新 CRL。
cmdInitOcsp	private static void cmdInitOcsp(Scanner sc, String baseDir) throws Exception	initOCSP：无参时为证书库中全部 R/C 证书签发 OCSP 签名证书。
cmdInitTimestamp	private static void cmdInitTimestamp(Scanner sc, String baseDir) throws Exception	initTIMSTAMP：无参时为全部根证书签发时间戳证书。
cmdVerifyCert	private static void cmdVerifyCert(Scanner sc, String baseDir) throws Exception	initTIMSTAMP：无参时为全部根证书签发时间戳证书。
doInit	public static String doInit(String baseDir, String dn, String bit, String hash, String algo) throws Exception	init：备份 + 模板 + 签发根证书 + 全量 OCSP + 全量 TSA。
doMakeRoot	public static String doMakeRoot(String baseDir, String dn, String bit, String hash, String algo) throws Exception	makeroot：签发根证书（签发后自动刷新 CRL），organizationIdentifier 扩展用旧版 OID。
doMakeRoot	public static String doMakeRoot(String baseDir, String dn, String bit, String hash, String algo, String evoid) throws Exception	makeroot：可指定 organizationIdentifier 扩展 OID 版本（new=2.23.140.3.1 / old=1.3.6.1.4.1.57264.1.8）。
doList	public static String doList(String baseDir) throws Exception	list：列出证书库（返回表格文本）。
doSignCert	public static String doSignCert(String baseDir, String dn, String bit, String hash, String keyUsage, String exUsage, String isCa, String days, String dns, String ips, String issuer, String algo) throws Exception	signCERT：签发证书（普通模式 12 参数，对齐 Go 版 MODML[1..12]）。
doSignCert	public static String doSignCert(String baseDir, String dn, String bit, String hash, String keyUsage, String exUsage, String isCa, String days, String dns, String ips, String issuer, String algo, String pathLen, String policy) throws Exception	signCERT 扩展版：pathLen=中间CA路径长度(null/0-9)；policy=终端证书策略OID(逗号分隔，勾选才写入)。
doRevoke	public static String doRevoke(String baseDir, String serial, String reason) throws Exception	RevokeCERT：吊销证书并刷新 CRL。
doRestore	public static String doRestore(String baseDir, String serial) throws Exception	RestoreCERT：恢复证书（撤销吊销，R→V、原因 0）并刷新 CRL。
doSignCrl	public static String doSignCrl(String baseDir) throws Exception	signCRL：刷新全部 CRL。
doInitOcsp	public static String doInitOcsp(String baseDir, String algo) throws Exception	initOCSP：为证书库中全部 R/C 证书签发 OCSP 签名证书。 @param algo 指定算法（RSA/ECC/SM2）；空串表示按各证书自身算法
doInitTimestamp	public static String doInitTimestamp(String baseDir, String hash) throws Exception	initTIMSTAMP：为全部根证书签发时间戳证书。 @param hash 指定哈希（SHA1/SHA256/SM3）；空串表示按各根证书算法全量
makeTsa	private static void makeTsa(String baseDir, String hash, String algo, String rootSerial) throws Exception	—
doVerifyCert	public static String doVerifyCert(String baseDir, String target) throws Exception	verifyCERT：验证证书，返回完整输出。

MakeRoot —— MakeRoot.java

常量

常量	值	说明
NOT_BEFORE	"2001-07-19 15:30:00"	证书生效时间（Go 版固定值）
NOT_AFTER	"2099-07-19 15:30:00"	证书过期时间（Go 版固定值）
CABF_ORG_ID_OID_OLD	"1.3.6.1.4.1.57264.1.8"	旧版扩展 OID：2019 SC27 ballot（CABF IANA 57264 arc）。
CABF_ORG_ID_OID_NEW	"2.23.140.3.1"	新版扩展 OID：EVG 2.0.1 / BR（CA/B Forum 自有 arc 2.23.140.3.1）。

方法一览（15 个）

方法	签名	说明
buildSerial	private static BigInteger buildSerial(String baseDir)	生成 128 位安全随机正序列号（十六进制小写，查 CERTS.txt 去重）。
existingSerials	private static java.util.Set<String> existingSerials(String baseDir)	读取 CERTS.txt 中已登记的序列号集合（十六进制小写），避免新序列号碰撞。
main	public static void main(String[] args) throws Exception	读不到视为空集合，随机序列号碰撞概率可忽略
run	public static void run(String baseDir, String[] args) throws Exception	供 ADMIN 内部调用：指定工作目录签发根证书。
parseKeyBits	private static int parseKeyBits(String arg, String keyType)	解析并归一化密钥位数（规则与 Go 版完全一致）。
buildSubject	private static X500Name buildSubject(String dn)	解析 DN 字符串为 X500Name（RDN 顺序对齐 Go 版 pkix.Name：C,O,OU,L,S,STREET,PostalCode,SERIALNUMBER,CN）。
dnOrder	private static int dnOrder(String key)	统一 DN 属性排序权重（X.500/RFC 规范顺序，从根到叶）：C→ST→L→STREET→PostalCode→O→OU→SERIALNUMBER→CN， EV 注册字段（jurisdiction* 注册市/省/国、organizationIdentifier、emailAddress）随后，自定义 OID 最后。
addDnField	private static void addDnField(X500NameBuilder builder, String key, String value)	按 key（短名或 OID）添加 RDN，非标准 key 直接按 OID 解析。
isStandardDnKey	private static boolean isStandardDnKey(String key)	标准 DN 短名（Go 版 pkix.Name 支持的固定字段）。
addIfPresent	private static void addIfPresent(X500NameBuilder builder, ASN1ObjectIdentifier oid, String value)	标准 DN 短名（Go 版 pkix.Name 支持的固定字段）。
addAuthorityInfoAccess	private static void addAuthorityInfoAccess(JcaX509v3CertificateBuilder cb, String ocspUrl, String caIssuerUrl) throws Exception	添加 AuthorityInfoAccess 扩展（OCSP + caIssuers）。
addCrlDistributionPoints	private static void addCrlDistributionPoints(JcaX509v3CertificateBuilder cb, String crlUrl) throws Exception	添加 CRL 分发点扩展（含 .crl 与 .der.crl 两个 URL，对齐 Go 版）。
getDnField	private static String getDnField(String dn, String key)	从逗号分隔 DN 中提取指定 key（标准名或 OID）的值，未找到返回 null。
addCABFOrganizationIdentifier	private static void addCABFOrganizationIdentifier(JcaX509v3CertificateBuilder cb, String orgId, String extOid)	—
buildCpsPolicies	private static org.bouncycastle.asn1.ASN1Primitive buildCpsPolicies(String cpsUrl, String cpsText) throws Exception	—

MakeCert —— MakeCert.java

常量

常量	值	说明
MOD_TIME	DateTimeFormatter.ofPattern("yyyy/MM/dd-HH:mm:ss")	—
EXT_OIDS	{ {"12", "1.3.6.1.4.1.311.2.1.22"}, {"13", "1.3.6.1.4.1.311.61.1.1"}, {"14", "1.3.6.1.4.1.311.10.3.5"}, {"15", "1.3.6.1.	ExtKeyUsage 12~46 自定义 OID（对齐 Go 版 makecert switch）

方法一览（40 个）

方法	签名	说明
setQuiet	public static void setQuiet(boolean q)	设置安静模式（服务进程内调用，执行完应恢复 false）。
out	private void out(String msg)	生成结果输出：控制台（非安静模式）+ 操作日志（始终写入，保证服务模式有据可查）。
main	public static void main(String[] args) throws Exception	生成结果输出：控制台（非安静模式）+ 操作日志（始终写入，保证服务模式有据可查）。
run	public static void run(String baseDir, String[] a) throws Exception	供 ADMIN 内部调用：指定工作目录签发。
run	private void run(String[] a) throws Exception	供 ADMIN 内部调用：指定工作目录签发。
resolveIssuer	private String resolveIssuer(String id)	---- 输出文件 ---- ========== 以下为辅助方法 ==========

方法	签名	说明
checkIssuerUsable	private void checkIssuerUsable(X509Certificate issuerCert, String issuerId) throws Exception	签发前校验颁发者可用性（PKI-009/PKI-010）：有效期、CA 属性、keyCertSign、吊销状态。"文件存在"不能等于"允许作为 CA 继续签发"。
defaultBit	private int defaultBit(String algo)	文件证书序列号必须与台账一致（防止文件与台账编号错位）
parseBit	private int parseBit(String arg, String algo)	文件证书序列号必须与台账一致（防止文件与台账编号错位）
generateKeyPair	private KeyPair generateKeyPair(String algo, int bit) throws Exception	FIPS 186-4：L 取 1024/2048/3072 三档 Ed25519 固定 256，忽略输入
resolveSignHash	private String resolveSignHash(String[] a, String rootAlgo, boolean isOcsp, boolean isTsa, String tsaHash)	决定证书签名算法哈希（对齐 Go 版 roothash 逻辑）。
buildSubject	private X500Name buildSubject(String dn)	—
dnOrder	private static int dnOrder(String key)	统一 DN 属性排序权重（X.500/RFC 规范顺序，从根到叶）：C→ST→L→STREET→PostalCode→O→OU→SERIALNUMBER→CN， EV 注册字段（jurisdiction* 注册市/省/国、organizationIdentifier、emailAddress）随后，自定义 OID 最后。
addDnField	private static void addDnField(X500NameBuilder builder, String key, String value)	按 key（短名或 OID）添加 RDN，非标准 key 直接按 OID 解析。
isStandardDnKey	private static boolean isStandardDnKey(String key)	标准 DN 短名（Go 版 pkix.Name 支持的固定字段）。
copySubject	private void copySubject(X500NameBuilder b, X509Certificate issuerCert, String cn, String ou)	标准 DN 短名（Go 版 pkix.Name 支持的固定字段）。
oidOf	private static ASN1ObjectIdentifier oidOf(String key)	使用新 CN 使用新 OU
addIfPresent	private static void addIfPresent(X500NameBuilder builder, ASN1ObjectIdentifier oid, String value)	—
extractCn	private static String extractCn(X509Certificate cert)	—
parseValidity	private Date[] parseValidity(String txt) throws Exception	—
buildSerial	private BigInteger buildSerial(boolean isCa)	—
existingSerials	private java.util.Set<String> existingSerials()	读取 CERTS.txt 中已登记的序列号集合（十六进制小写），避免新序列号碰撞。
goKeyUsageToBc	private static int goKeyUsageToBc(int go)	Go 版 x509.KeyUsage 位值 → BC KeyUsage 位值。 <p>Go：DigitalSignature=1 ContentCommitment=2 KeyEncipherment=4 DataEncipherment=8 KeyAgreement=16 CertSign=32 CRLSign=64 EncipherOnly=128。BC：digitalSignature=128 nonRepudiation=64
parseKeyUsage	private int parseKeyUsage(String mode)	—
builtinEku	private static KeyPurposeId builtinEku(int num)	自定义组合：支持 "C5" 前缀形式（WebAdmin 悬浮窗组合）与纯数字位值 （如 11=数字签名1+密钥加密4+数据加密8），直接按 Go 位值解析
extOid	private static String extOid(int num)	—
isValidOid	private static boolean isValidOid(String s)	OID 合法格式校验：数字.数字.数字...（至少两段）
buildEkuSequence	private static DERSequence buildEkuSequence(List<KeyPurposeId> eku, List<ASN1ObjectIdentifier> unknown)	OID 合法格式校验：数字.数字.数字...（至少两段）
addAuthorityInfoAccess	private void addAuthorityInfoAccess(JcaX509v3CertificateBuilder cb, String ocspUrl, String caIssuerUrl) throws Exception	—
addCrlDistributionPoints	private void addCrlDistributionPoints(JcaX509v3CertificateBuilder cb, String crlUrl) throws Exception	—
addSan	private void addSan(JcaX509v3CertificateBuilder cb, String dns, String ips) throws Exception	—
buildCpsPolicies	private CertificatePolicies buildCpsPolicies(String[] oids, String cpsUrl, String cpsText) throws Exception	构建 CPS 证书策略扩展（参考 smplus：CertificatePolicies + PolicyInformation + PolicyQualifierInfo 标准类）。 oids[0] 带 CPS URI + UserNotice 限定符，oids[1..] 为纯 OID 策略。

方法	签名	说明
buildCertBuilder	private JcaX509v3CertificateBuilder buildCertBuilder(X500Name issuer, BigInteger serial, Date notBefore, Date notAfter, X500Name subject, java.security.PublicKey pubKey, boolean isCa, Integer pathLen, int keyUsage, byte[] issuerSki, byte[] userSki, List<KeyPurposeId> eku, List<ASN1ObjectIdentifier> unknownEku, boolean isTsa, boolean isOcsp, String ocspUrl, String crtUrl, String crlUrl, CertificatePolicies certPolicies, String dns, String ips, boolean addPoison) throws Exception	构建证书构建器并添加全部扩展（CPS 策略 / AIA / CRL / SAN / EKU / 专属扩展）。 @param addPoison true = RFC 6962 §3.1 poison 扩展（critical，extnValue = OCTET STRING 05 00） —— 仅用于 SCT 预证书；false = 普通证书
writeOutputs	private void writeOutputs(X509Certificate cert, KeyPair kp, String algo, String serialHex, boolean isCa, boolean isOcsp, boolean isTsa, String issuerId, String tsaHash) throws Exception	RFC 6962 §3.1 poison：critical，extnValue = OCTET STRING 包含 ASN.1 NULL(05 00) ========== 输出 ==========
removeCertsLine	private synchronized void removeCertsLine(String serial) throws Exception	从 CERTS.txt 删除指定序列号行（重跑替换旧登记）。 保留 # 注释/空行（与 CertsDB 语义一致）； PemIO.atomicWrite 原子替换，避免 TSA/OCSP 重跑时并发或中断导致 CERTS.txt 半截。
concat	private static byte[] concat(byte[] a, byte[] b)	========== 工具 ==========
slice	private static byte[] slice(byte[] a, int from, int len)	========== 工具 ==========
longTo8Bytes	private static byte[] longTo8Bytes(long v)	========== 工具 ==========
contains	private static boolean contains(byte[] a, byte target)	—
hexToBytes	private static byte[] hexToBytes(String hex)	—

RootGetCrl —— RootGetCrl.java

常量

常量	值	说明
MOD_TIME	DateTimeFormatter.ofPattern("yyyy/MM/dd-HH:mm:ss")	RootGetCrl —— 签发吊销列表 CRL（对应 Go 版 cmd/rootgetcrl）。 <p>遍历 CERTS.txt 中类型为 R（根）或 C（中间）的证书，对每一个颁发者： - R 类型：从 PKI/ROOT/序列号.crt|.key 加载颁发者密钥对； - C 类型：从 PKI/CA/序列号.crt|.key 加载颁发者密钥对； 收集 CERTS.txt 中全部状态为

方法一览（9 个）

方法	签名	说明
main	public static void main(String[] args) throws Exception	—
run	public static void run() throws Exception	供其它程序（如 MakeRoot / Admin）内部调用。
run	public static void run(String baseDir) throws Exception	供 ADMIN 内部调用：指定工作目录刷新 CRL。
generateCrl	private static void generateCrl(CertsDB.CertEntry issuer, String rootDir, String caDir, String webCrlDir, CertsDB db) throws Exception	—
revokedCount	private static int revokedCount(CertsDB.CertEntry issuer, List<CertsDB.CertEntry> all)	—
nextCrlNumber	private static synchronized BigInteger nextCrlNumber(String webCrlDir, String issuerSerial, String akiHex) throws Exception	读取/递增 CRL 顺序号（RFC 5280 5.2.3）。 历史序号存储于 <webCrlDir>/num.txt，行格式（# 开头为注释）： <颁发者证书序列号> <颁发者密钥标识符hex> <CRL顺序号> 例如: A000001 ABCDABCDABCDABCD... 1 未登记过该颁发者时从 1 起始；每次发布递增 1。读改写在同一个 synchronized 块内完成，避免并发
parseModTime	private static Date parseModTime(LocalDateTime t)	CERTS.txt 内部时间（本地时间）转 Date。
certHashFromSigAlg	private static String certHashFromSigAlg(X509Certificate cert)	由证书签名算法 OID 推导哈希算法。
pssHashFromSigParams	private static String pssHashFromSigParams(X509Certificate cert)	从 RSASSA-PSS 证书签名参数（RSASSAPSSparams）解析哈希算法名。

CertVerify —— CertVerify.java

常量

常量	值	说明
MAX_HTTP_SIZE	5 * 1024 * 1024	HTTP 响应最大字节数（防恶意证书 AIA/CRL/OCSP 拉取超大响应，P2-01）。
SCT_EXT_OID	"1.3.6.1.4.1.11129.2.4.2"	SCT 扩展 OID（RFC 6962 3.2）。

方法一览（62 个）

方法	签名	说明
main	public static void main(String[] args) throws Exception	—
run	public static void run(String filename) throws Exception	供 ADMIN verifyCERT / WebAdmin /api/verify 内部调用。
parseCert	private static X509Certificate parseCert(byte[] data) throws Exception	解析证书字节：自动识别 PEM / DER。
run	private static void run(Path p) throws Exception	解析证书字节：自动识别 PEM / DER。
sigAlgDisplay	private static String sigAlgDisplay(String jcaName)	JCA 签名算法名转 Go 风格显示（SHA256WITHRSA → SHA256-RSA、SHA256WITHECDSA → SHA256-ECDSA）。
getHashType	private static String[] getHashType(X509Certificate cert)	JCA 签名算法名转 Go 风格显示（SHA256WITHRSA → SHA256-RSA、SHA256WITHECDSA → SHA256-ECDSA）。
getHashType	private static String[] getHashType(String sigAlgName)	RSASSA-PSS：BC 的 getSigAlgName() 通常为 "RSASSA-PSS"，须从签名参数解析哈希
describePublicKey	private static String describePublicKey(PublicKey pub)	—
printExtKeyUsage	private static void printExtKeyUsage(X509Certificate cert) throws Exception	—
ekuName	private static String ekuName(String oid)	—
printSan	private static void printSan(X509Certificate cert) throws Exception	---- Apple ----
printAiaAndCrl	private static void printAiaAndCrl(X509Certificate cert) throws Exception	—
printPolicies	private static void printPolicies(X509Certificate cert) throws Exception	—
policyName	private static String policyName(String oid)	—
printExtensions	private static void printExtensions(X509Certificate cert) throws Exception	打印扩展（含 CT SCT 列表与 CT Log 公钥）。
printOneExtension	private static void printOneExtension(X509Certificate cert, String oid, boolean printed, boolean isCritical) throws Exception	—
printSctList	private static void printSctList(X509Certificate cert, byte[] value)	解析并打印 SCT 列表（RFC 6962 TLS 编码）。 <p>结构（与权威解析工具 X509JsonConverter.parseSCTList/parseSCT 一致）： SCTList = uint16 totalLen + [uint16 sctLen + SerializedSCT]*； SerializedSCT = version(1) + log_id(32) + timestamp(8) + extensions(
sigName	private static String sigName(int hash, int type)	一条都未解析出来（格式异常）：退回打印原始 hex
verifySignature	private static boolean verifySignature(X509Certificate cert, PublicKey pub)	用给定公钥验证证书签名。
chainConstraintsOk	private static String[] chainConstraintsOk(X509Certificate child, X509Certificate parent)	PKIX 链约束检查（不含有效期——离线/长期验证：证书过期不影响信任关系）。 a) child.issuer DN == parent.subject DN b) child AKI(若存在) == parent SKI c) parent KeyUsage(若存在) 必须含 keyCertSign d) parent 必须是 CA；child 为 CA 时 parent pathLenConstraint 不能为 0 @return
akiMatches	private static boolean akiMatches(X509Certificate child, X509Certificate parent)	child AKI 是否与 parent SKI 匹配（两者都存在时）。
fetchIssuerSilent	private static X509Certificate fetchIssuerSilent(X509Certificate cert)	静默拉取颁发者证书（不打印证书链/信任状态），供 SCT 验签等内部使用。 优先 AIA caIssuers，AIA 缺失/失败时回退本地 PKI/ROOT、PKI/CA、PKI/MY、java cacerts。

方法	签名	说明
fetchIssuer	private static X509Certificate fetchIssuer(X509Certificate cert)	获取颁发者证书：优先 AIA caIssuers URL；AIA 缺失/下载失败时回退本地 PKI/ROOT、PKI/CA、PKI/MY、java cacerts（按 Subject DN + 验签匹配）。 返回直接颁发者用于验签，并打印证书链。
printChain	private static void printChain(X509Certificate issuer)	递归打印证书链：中间证书 CA --> SKI，根证书 Root --> SKI，并输出信任状态。 每级都做 child.verify(parent.publicKey) 逐级验签，且上级必须为 CA （BasicConstraints >= 0）；上级优先 AIA，AIA 缺失/失败时回退本地 PKI/ROOT、PKI/CA、PKI/MY、java cacerts（PKI-004/PKI-005）。
findLocalIssuer	private static X509Certificate findLocalIssuer(X509Certificate cert)	本地颁发者查找：按证书 Issuer DN 匹配 PKI/ROOT、PKI/CA、PKI/MY 及 java cacerts，并用候选公钥验签当前证书；无验签成功者退回第一个 DN 匹配（用于链显示）。
findLocalIssuerInCacerts	private static X509Certificate findLocalIssuerInCacerts(X509Certificate cert, String issuerDn)	在 JAVA cacerts 中查找能验证当前证书签名的同 DN 证书。
isInTrustAnchor	private static boolean isInTrustAnchor(X509Certificate cert)	实验 TrustAnchor 检查：证书公钥是否存在于 PKI/ROOT.crt（按 SPKI 字节匹配）。 <p>自签名验证通过只证明签名有效；是否属于当前环境的信任锚， 以 PKI/ROOT 目录中是否登记该根为准（实验型 PKI 的信任模型）。
inJavaCacerts	private static boolean inJavaCacerts(X509Certificate cert)	JAVA cacerts 信任锚检查：证书公钥（SPKI）是否存在于 {@code <java.home>/lib/security/cacerts}（JKS/PKCS12，默认密码 changeit）。
isSelfSigned	private static boolean isSelfSigned(X509Certificate c)	判断证书是否为自签名（Subject == Issuer）。
subjectKeyIdHex	private static String subjectKeyIdHex(X509Certificate c)	证书 SKI 纯净 hex（剥离双层 OCTET STRING 包装）。
aiaCaIssuers	private static List<String> aiaCaIssuers(X509Certificate cert) throws Exception	证书 SKI 纯净 hex（剥离双层 OCTET STRING 包装）。
checkCrl	private static void checkCrl(X509Certificate cert)	吊销状态检查 ========== ========== ==========
checkOcsp	private static void checkOcsp(X509Certificate cert)	—
ocspResponderKey	private static PublicKey ocspResponderKey(org.bouncycastle.cert.ocsp.BasicOCSPResp basic, X509Certificate issuerCert) throws Exception	取 OCSP 响应者公钥（responder 授权，RFC 6960 §4.2.2.3）： 优先使用 BasicOCSPResp 内嵌的 OCSP 签名证书；无内嵌证书时 视为颁发者直接应答，使用颁发者公钥。
buildOcspRequest	private static byte[][] buildOcspRequest(X509Certificate cert, X509Certificate issuerCert) throws Exception	构造 SHA1 CertID 的 OCSP 请求（含 Nonce 扩展）。返回 {请求字节, nonce}。
crlUrls	private static List<String> crlUrls(X509Certificate cert) throws Exception	—
ocspUrls	private static List<String> ocspUrls(X509Certificate cert) throws Exception	—
printHashes	private static void printHashes(X509Certificate cert, PublicKey pub) throws Exception	哈希打印 ========== ========== ==========
rfcSha1	private static byte[] rfcSha1(PublicKey pub) throws Exception	RFC5280 方法1 的密钥标识（RSA：PKCS#1 位串；ECC/SM2：未压缩点）。
copyRightAligned	private static void copyRightAligned(byte[] src, byte[] dst, int offset, int size)	未压缩点：04 || X || Y（固定长度，X/Y 按曲线字节数填充）
httpGet	private static byte[] httpGet(String url, int timeoutMs)	HTTP 响应最大字节数（防恶意证书 AIA/CRL/OCSP 拉取超大响应，P2-01）。
readAll	private static byte[] readAll(InputStream in) throws IOException	网络策略（P2-02）：只允许 HTTP / HTTPS，拒绝 file: 等其它协议
unwrapOctets	private static byte[] unwrapOctets(byte[] extValue)	去掉扩展值的 OCTET STRING 外层。
subjectKeyIdValue	private static byte[] subjectKeyIdValue(byte[] extValue)	SKI 扩展值 → 纯净字节：extValue(04..) → SubjectKeyIdentifier(04 14 ..) → 20 字节。
authorityKeyIdValue	private static byte[] authorityKeyIdValue(byte[] extValue)	AKI 扩展值 → keyIdentifier 纯净字节：用 BC 解析 AuthorityKeyIdentifier 结构。
chunks	private static byte[][] chunks(byte[] data)	AKI 扩展值 → keyIdentifier 纯净字节：用 BC 解析 AuthorityKeyIdentifier 结构。
indexOf	private static int indexOf(byte[] data, byte[] needle)	—

方法	签名	说明
hex	private static String hex(byte[] data)	—
sha256	private static byte[] sha256(byte[] data) throws Exception	—
bitsToString	private static String bitsToString(boolean[] bits)	—
parseSctListData	private static List<SctInfo> parseSctListData(byte[] value)	解析 TLS 编码的 SCT 列表。SCTList = uint16 totalLen + [uint16 sctLen + SerializedSCT]*。
sctStatus	private static String sctStatus(byte[] logId, int version, long timestamp, byte[] extensions, int hashAlg, int sigAlg, byte[] signature, Map<String, String[]> logKeys, X509Certificate issuer, byte[] issuerKeyHash, byte[] tbsNoSct, byte[] certDer)	计算单条 SCT 的验证状态（并入 SCT列表详细输出）。 状态三态：签名有效 / 签名无效 / 未知日志，无法验证。
buildSctInput	private static byte[] buildSctInput(SctInfo sct, byte[] issuerKeyHash, byte[] tbsNoSct, byte[] certDer, boolean precert) throws IOException	构造 RFC 6962 3.2 的签名输入（precert=true 用 PreCert，否则用完整证书）。
tbsWithoutSctExt	private static byte[] tbsWithoutSctExt(X509Certificate cert) throws Exception	从证书 TBS 中移除 SCT 扩展（RFC 6962 3.1 PreCert.tbs_certificate）。
verifySctSignature	private static boolean verifySctSignature(byte[] input, SctInfo sct, byte[] spkiDer) throws Exception	用日志公钥（SPKI DER）验证 SCT 签名。hash/sig 为 TLS 算法枚举（RFC 5246 / RFC 8422）。
tlsHashName	private static String tlsHashName(int hash)	TLS HashAlgorithm 枚举 → JCA 哈希数字后缀。
ecdsaRawToDer	private static byte[] ecdsaRawToDer(byte[] raw) throws Exception	TLS ECDSA 定长 r||s → DER SEQUENCE { INTEGER r, INTEGER s }。
loadCtLogKeys	private static Map<String, String[]> loadCtLogKeys()	Load CT log public keys: Google log_list v3 + local PKI/CTLOG/modpkica_ct_list.json (merged). Returns log_id(base64) -> [description, key(base64 SPKI)].
writeU16	private static void writeU16(ByteArrayOutputStream bos, int v)	—
writeU24	private static void writeU24(ByteArrayOutputStream bos, int v)	—
writeU64	private static void writeU64(ByteArrayOutputStream bos, long v)	—
pssHashName	private static String pssHashName(X509Certificate cert)	从 RSASSA-PSS 证书签名参数解析哈希名（大写，SHA3 带连字符，供 JCA 算法名使用）。

Auto —— Auto.java

常量

常量	值	说明
MOD_TIME	DateTimeFormatter.ofPattern("yyyy/MM/dd-HH:mm:ss")	CERTS.txt 行时间格式
CERTS_TEMPLATE	"# 序列号 类型 状态 状态原因编号 签发时间/吊销时间\r\n" + "# ROOT/CA/END V正常R吊销N未知 0-9 例如 2000/06/17-12:12:12\r\n"	CERTS.txt 模板内容（对齐 Go 版 echo 出的字节内容）

方法一览（10 个）

方法	签名	说明
main	public static void main(String[] args) throws Exception	CERTS.txt 模板内容（对齐 Go 版 echo 出的字节内容）
now	private static String now()	当前真实时间（登记/吊销时间戳，用户确认不偏移）。
runInit	public static void runInit(String baseDir) throws Exception	供 ADMIN 内部调用：初始化环境（备份 + 模板）。
revoke	public static void revoke(String baseDir, String serial, String reason) throws Exception	供 ADMIN 内部调用：吊销证书。
restore	public static void restore(String baseDir, String serial) throws Exception	供 ADMIN/WebAdmin 内部调用：恢复证书（R→V、原因 0、时间刷新为当前时间）。
initEnv	private static void initEnv(String baseDir) throws Exception	auto init：清理证书目录旧文件、备份 CERTS.txt 并写入模板。
revokeCert	private static void revokeCert(String baseDir, String serial, String reason) throws Exception	auto revoke：把 CERTS.txt 中指定序列号记录改为 R 状态。

方法	签名	说明
restoreCert	private static void restoreCert(String baseDir, String serial) throws Exception	auto restore：把 CERTS.txt 中指定序列号记录改为 V 状态、原因 0（撤销吊销）。
addCert	private static void addCert(String baseDir, String serial, String type, String status, String time, String issuerId, String algo) throws Exception	auto newcert：追加一条证书记录。
atomicWrite	private static void atomicWrite(Path p, String content) throws Exception	原子写 CERTS.txt（tmp → fsync → .bak → move），与 CertsDB.rewriteAll 同策略。

各主程序运行示例

```
MAIN       java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.MAIN
Admin      java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.Admin initTIMESTAMP
           ... Admin verify 113522cb2.crt / Admin initOCSP / Admin initTIMESTAMP / Admin rekey ...
MakeRoot   java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.MakeRoot "CN=Root,O=Org,C=CN" 2048 sha256 RSA
MakeCert   java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.MakeCert "CN=www.test.com" 2048 sha256 1 1 0 1 "www.tes
RootGetCrl java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.RootGetCrl
CertVerify java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.CertVerify cert.crt
Auto       java "-Dfile.encoding=UTF-8" -cp "out;BC185\*;lib\*" modpkica.Auto revoke 1234567890 1
```

第四章 工具类详解（16 个）

AppEnv —— AppEnv.java

方法一览（1 个）

方法	签名	说明
getExeNames	public static String[] getExeNames()	返回 MODPKICA 全部可执行应用名称（Java 版不带 .exe 后缀）。 @return 长度为 7 的应用名称数组

AppPaths —— AppPaths.java

方法一览（1 个）

方法	签名	说明
baseDir	public static String baseDir()	返回 MODPKICA 根目录（其下应有 PKI 目录）。

Bc —— Bc.java

方法一览（1 个）

方法	签名	说明
init	public static void init()	注册 BC Provider（幂等，可重复调用）。

CertUtil —— CertUtil.java

常量

常量	值	说明
HASH_NAMES	new HashSet<>()	摘要名称集合，便于归一化
signer	new SM2Signer()	SM2 专用 ContentSigner。 <p>BC 的 JcaContentSignerBuilder("SM3withSM2") 不保证按 GB/T 32918 计算 ZA， 这里直接用 SM2Signer + ParametersWithID（默认用户ID），对 getOutputStream() 收到的 TBS 原文计算 e=SM3(ZA||TBS)，输出 DER(r,s)。
buffer	new ByteArrayOutputStream()	SM2 专用 ContentSigner。 <p>BC 的 JcaContentSignerBuilder("SM3withSM2") 不保证按 GB/T 32918 计算 ZA， 这里直接用 SM2Signer + ParametersWithID（默认用户ID），对 getOutputStream() 收到的 TBS 原文计算 e=SM3(ZA||TBS)，输出 DER(r,s)。

方法一览（18 个）

方法	签名	说明
generateRsaKeyPair	public static KeyPair generateRsaKeyPair(int bits) throws Exception	生成 RSA 密钥对。
generateEccKeyPair	public static KeyPair generateEccKeyPair(int bits) throws Exception	生成 ECC 密钥对（224/256/384/521，对应 secp224r1/secp256r1/secp384r1/secp521r1）。
generateSm2KeyPair	public static KeyPair generateSm2KeyPair() throws Exception	生成 SM2 密钥对（sm2p256v1）。
generateDsaKeyPair	public static KeyPair generateDsaKeyPair(int bits) throws Exception	生成 DSA 密钥对（FIPS 186-4：1024/2048/3072 位，L=1024..3072 且为 64 的倍数）。 BC 的 DSA KeyPairGenerator 不接受任意非标准长度，按三档归一。
generateEd25519KeyPair	public static KeyPair generateEd25519KeyPair() throws Exception	生成 Ed25519 密钥对（RFC 8032，固定 256 位，无密钥位数参数）。

方法	签名	说明
generatePqcKeyPair	public static KeyPair generatePqcKeyPair(String spec) throws Exception	生成 ML-DSA（FIPS 204）PQC 密钥对。 @param spec 规格字符串，接受 "ML-DSA-44"/"ML-DSA-65"/"ML-DSA-87"（也兼容 "44"/"65"/"87" 等）
normalizePqcSpec	public static String normalizePqcSpec(String spec)	归一化 ML-DSA 规格为 BC 算法名（ML-DSA-44/65/87），非法输入回退 ML-DSA-65。
getSignatureAlgOid	public static ASN1ObjectIdentifier getSignatureAlgOid(String keyType, String hash)	根据颁发者密钥类型与摘要返回签名算法 OID。 @param keyType "RSA" / "ECC" / "SM2" / "DSA" / "EdDSA" / "PQC" @param hash 摘要名（RSA/ECC/DSA 时使用；SM2 忽略固定 SM3；EdDSA/PQC 忽略）
normalizeHash	private static String normalizeHash(String hash)	归一化摘要名为 SHA1/SHA224/SHA256/SHA384/SHA512/SHA3-224/SHA3-256/SHA3-384/SHA3-512/SM3。 SHA3 系列必须保留连字符（JCA 算法名 SHA3-256withRSA），其余沿用旧逻辑去掉连字符。
readJcaPrivateKey	public static PrivateKey readJcaPrivateKey(String path) throws Exception	读取 PEM 私钥文件并转为 JCA PrivateKey（RSA/ECC/SM2 通用）。 <p>内部通过 {@code JcaPEMKeyConverter} 把 BC 的 AsymmetricKeyParameter 转为 JCA PrivateKey，供 JcaContentSignerBuilder 等使用。
createContentSigner	public static ContentSigner createContentSigner(PrivateKey key, String keyType, String hash) throws OperatorCreationException	根据颁发者私钥与算法类型构造 ContentSigner。 SM2 时返回自定义 {@link SM2ContentSigner}。 @param key 颁发者私钥（JCA PrivateKey） @param keyType "RSA" / "ECC" / "SM2" @param hash 摘要名
getAlgorithmIdentifier	public AlgorithmIdentifier getAlgorithmIdentifier()	外层 ParametersWithID，内层 ParametersWithRandom
getOutputStream	public OutputStream getOutputStream()	外层 ParametersWithID，内层 ParametersWithRandom
getSignature	public byte[] getSignature()	外层 ParametersWithID，内层 ParametersWithRandom
getSubjectKeyIdentifier	public static byte[] getSubjectKeyIdentifier(PublicKey pubKey) throws Exception	计算公钥的 SubjectKeyIdentifier：对公钥 SubjectPublicKeyInfo 中的 subjectPublicKey 位串做 SHA-1（RFC5280 方法 1）。
buildSelfSignedRoot	public static X509Certificate buildSelfSignedRoot(X500Principal subject, KeyPair kp, BigInteger serial, Date notBefore, Date notAfter, String keyType, String hash) throws Exception	生成自签名根证书。 @param subject 主体 DN @param kp 根密钥对 @param serial 序列号 @param notBefore 生效起 @param notAfter 生效止 @param keyType "RSA" / "ECC" / "SM2" @param hash 摘要名
buildSignedCert	public static X509Certificate buildSignedCert(X509Certificate issuerCert, PrivateKey issuerKey, X500Principal subject, PublicKey subjectPub, BigInteger serial, Date notBefore, Date notAfter, String keyType, String hash, boolean isCa, String crlUrl, String ocspUrl, String caIssuerUrl, String cpsUri, String userNotice) throws Exception	—
buildCpsPolicy	private static PolicyInformation buildCpsPolicy(String cpsUri, String userNotice)	构造 CertificatePolicies 扩展值（PolicyInformation + CPS URI + UserNotice）。 参考 Go 版 internal/cps/cps.go。

CertsDB —— CertsDB.java

常量

常量	值	说明
TIME_FMT	DateTimeFormatter.ofPattern("yyyy/MM/dd-HH:mm:ss")	时间列的解析格式
entries	new CopyOnWriteArrayList<>()	内存中缓存的全部台账条目（CopyOnWriteArrayList：读多写少，遍历永不抛 CME）
preservedLines	new ArrayList<>()	原文件中 # 注释行与空行（用户"删除证书=行首 # 注释"的语义载体）。 仅 load 与 rewriteAll 在锁内访问；rewriteAll 时原样保留，避免重写抹掉注释。
FILE_LOCK	new Object()	进程内互斥（配合跨进程文件锁，见 withLock）。

方法一览（15 个）

方法	签名	说明
withLock	private void withLock(Runnable action)	跨进程 + 进程内双重互斥：通过 CERTS.txt.lock 文件锁实现（P2-02）。 动作内部抛出的 RuntimeException（含 UncheckedIOException）原样上抛。
load	public void load(String baseDir)	加载 <baseDir>/PKI/CERTS.txt。 @param baseDir MODPKICA 根目录
reload	public void reload()	重新从磁盘加载（外部修改后调用）。
findBySerial	public CertEntry findBySerial(String serial)	按序列号查找条目。 @param serial 证书序列号 @return 命中的条目，未命中返回 null
findByIssuerAndSerial	public CertEntry findByIssuerAndSerial(String issuerId, String serial)	按颁发者序列号 + 证书序列号查找（issuer+serial 才是全局唯一键）。 @param issuerId 颁发者序列号 @param serial 证书序列号 @return 命中的条目，未命中返回 null
findIssuerIdBySerial	public String findIssuerIdBySerial(String serial)	按序列号查找其颁发者序列号。 @param serial 证书序列号 @return 颁发者序列号；找不到时返回 "null"（与 Go 版字符串约定一致）
addEntry	public void addEntry(CertEntry e)	追加一条新条目到内存并写回 CERTS.txt 文件末尾。 内存 add 与文件写同在锁内，保证并发追加不会出现内存/磁盘不一致。 @param e 新条目
revokeCert	public void revokeCert(String serial, int reason, LocalDateTime time)	把指定序列号证书状态改为 R（吊销），更新原因与时间，并整文件重写。 内存采用"新对象替换"而非改字段，读方不会看到半更新状态。 @param serial 证书序列号 @param reason 吊销原因码 @param time 吊销时间
getAllRootAndCA	public List<CertEntry> getAllRootAndCA()	返回所有类型为 R（根）或 C（中间 CA）的条目。
getAllRevoked	public List<CertEntry> getAllRevoked()	返回所有状态为 R（吊销）的条目。
getAll	public List<CertEntry> getAll()	返回全部条目（只读快照，调用方修改不影响库内数据）。
cloneEntry	private static CertEntry cloneEntry(CertEntry e)	复制条目（吊销/恢复时以新对象替换旧对象，避免读方看到半更新字段）。
rewriteAll	private void rewriteAll()	把内存中全部条目按行重写回文件（P2-01：tmp → fsync → .bak → 原子替换，锁内执行）。 注释/空行（preservedLines）按原顺序写回最前，保证"行首 # 注释"的删除语义不被重写抹掉。
formatLine	private static String formatLine(CertEntry e)	把条目格式化为文件中的一行（不补换行）。
stripCrLf	private static String stripCrLf(String s)	去掉行尾 \r\n。

Config —— Config.java

常量

常量	值	说明
raw	new HashMap<>()	原始键值映射，便于以后扩展

方法一览（10 个）

方法	签名	说明
load	public final void load(String baseDir)	加载配置文件。WEBPORT 缺省时回退为 "80"。 @param baseDir MODPKICA 根目录
getCpsWithCid	public String getCpsWithCid(String certId)	返回把 {@code {CID}} 替换为 certId 后的 CPS URL。 @param certId 证书序列号（hex 字符串） @return 替换后的 CPS URL；cps 为空时原样返回空串
stripCrLf	private static String stripCrLf(String s)	去掉一行末尾可能残留的 \r / \n（跨平台读写兼容）。
getCrlUrl	public String getCrlUrl()	去掉一行末尾可能残留的 \r / \n（跨平台读写兼容）。
getCrtUrl	public String getCrtUrl()	去掉一行末尾可能残留的 \r / \n（跨平台读写兼容）。

方法	签名	说明
getOcspUrl	public String getOcspUrl()	去掉一行末尾可能残留的 \r / \n（跨平台读写兼容）。
getCps	public String getCps()	去掉一行末尾可能残留的 \r / \n（跨平台读写兼容）。
getCpsNonice	public String getCpsNonice()	---- getter / setter ----
getWebPort	public String getWebPort()	---- getter / setter ----
getRaw	public Map<String, String> getRaw()	---- getter / setter ----

CrlUtil —— CrlUtil.java

方法一览（3 个）

方法	签名	说明
generateCrl	public static byte[] generateCrl(X509Certificate caCert, PrivateKey caKey, List<RevokedCert> revoked, Date thisUpdate, Date nextUpdate, String keyType, String hash, BigInteger crlNumber) throws Exception	生成 CRL（DER 字节）。 @param caCert 签发 CA 证书 @param caKey CA 私钥 @param revoked 吊销列表（可为空列表，表示当前无吊销） @param thisUpdate 本次更新时间 @param nextUpdate 下次更新时间 @param keyType "RSA" / "ECC" / "SM2" @param hash 摘要名（RSA/ECC
generateCrlObject	public static X509CRL generateCrlObject(X509Certificate caCert, PrivateKey caKey, List<RevokedCert> revoked, Date thisUpdate, Date nextUpdate, String keyType, String hash, BigInteger crlNumber) throws Exception	生成 CRL 并返回 X509CRL 对象。
derToPem	public static String derToPem(byte[] der) throws Exception	把 DER 编码的 CRL 转成 PEM 文本（Type=X509 CRL）。

CtLog —— CtLog.java

常量

常量	值	说明
OID_POISON	"1.3.6.1.4.1.11129.2.4.3"	poison 扩展 OID（RFC 6962 §3.1）。
OID_SCT	"1.3.6.1.4.1.11129.2.4.2"	SCT 列表扩展 OID（RFC 6962 §3.2）。
PUBLIC_LIST_URL	"https://www.gstatic.com/ct/log_list/v3/log_list.json"	公网 CT 日志列表的远程地址。

方法一览（37 个）

方法	签名	说明
keysJsonPath	public static String keysJsonPath(String baseDir)	密钥库文件路径。
logsTxtPath	public static String logsTxtPath(String baseDir)	请求日志文件路径。
publicListPath	public static String publicListPath(String baseDir)	可信公网 CT 日志列表文件路径（Google CT log list v3）。
loadPublicList	public static synchronized List<PubLog> loadPublicList(String baseDir)	读取可信公网 CT 日志列表（log_list.json，Google log_list v3）。文件不存在返回空列表；state 为对象形式 {@code {"usable": {...}}} 时提取键名。 P2-12：改用 MiniJson DOM 解析（替代脆弱的正则提取）。
publicListTimestamp	public static synchronized String publicListTimestamp(String baseDir)	读取 log_list.json 的数据日期（log_list_timestamp）。文件不存在返回空串。
fetchPublicList	public static synchronized String fetchPublicList(String baseDir) throws Exception	从 gstatic 拉取最新 log_list.json 并保存到 PKI/CTLOG/log_list.json。返回结果描述。
loadKeys	public static synchronized List<LogKey> loadKeys(String baseDir)	读取全部日志密钥对。文件不存在返回空列表。P2-12：MiniJson DOM 解析。
grab	private static String grab(String seg, String field)	从 JSON 片段中提取字符串字段值。
saveKeys	public static synchronized void saveKeys(String baseDir, List<LogKey> keys) throws Exception	写回密钥库（v3 风格结构）。
esc	private static String esc(String s)	—

方法	签名	说明
generateKey	public static synchronized LogKey generateKey(String baseDir, String desc, String type) throws Exception	生成日志密钥对并追加写入密钥库（type：ECDSA-P256 / RSA-2048 / SM2-P256）。返回新密钥。
deleteKey	public static synchronized boolean deleteKey(String baseDir, String logIdB64)	按 log_id 删除密钥对。返回是否删除成功。
setState	public static synchronized boolean setState(String baseDir, String logIdB64, String state)	设置密钥状态：usable（启用）/ disable（暂停）。
pickLogKeys	public static List<LogKey> pickLogKeys(String baseDir, boolean sm2Precert)	按预证书类型随机选出参与 SCT 签名的日志密钥组（最多 3 组，仅 usable）：SM2 预证书 → 只选 SM2 密钥；RSA/ECDSA 预证书 → 只选 RSA 或 ECDSA 密钥。
isSm2Precert	public static boolean isSm2Precert(X509Certificate cert)	判断预证书是否 SM2 类型（签名算法 SM3withSM2，或公钥 named curve = SM2 曲线）。
signSctList	public static byte[] signSctList(String baseDir, byte[] tbsNoPoison, byte[] issuerKeyHash, List<LogKey> pool) throws Exception	用选定的一组日志密钥对同一 precert 签发 SCT，组装 SCTList（TLS 编码）。 @param baseDir MODPKICA 根目录 @param tbsNoPoison 预证书 TBS 删除 poison 扩展后的 DER（RFC 6962 §3.1） @param issuerKeyHash 颁发者公钥 SPKI 的 SHA-256（32 字节） @param pool
submitPrecert	public static byte[] submitPrecert(String baseDir, X509CertificateHolder precert, X509Certificate issuerCert, String port) throws Exception	将 PEM 预证书 + PEM 颁发者证书以 POST 提交到自身 MAIN /ct/add， 返回 SCTList（二进制，直接作为 1.3.6.1.4.1.11129.2.4.2 扩展值）。 MAIN 服务未启动（HTTP 失败）时回退本地签名，保证签发流程可用。
appendRequestLog	public static synchronized void appendRequestLog(String baseDir, String line)	追加一行 CT 请求日志。行格式： {@code SCT待签名数据SHA256 || 毫秒时间戳 || 使用者SKI || 颁发者AKI || 日志公钥ID(base64)}
readLogs	public static List<String> readLogs(String baseDir)	读取全部 CT 请求日志行（按行拆分，供 Web 面板只读展示）。
parsePemCerts	public static List<X509Certificate> parsePemCerts(byte[] pemBytes) throws Exception	从 PEM 文本中解析全部证书（按块顺序）。
pemCert	public static String pemCert(byte[] der)	证书 PEM 序列化。
tbsWithoutPoison	public static byte[] tbsWithoutPoison(X509Certificate cert) throws Exception	从证书 TBS 中删除 poison 扩展（RFC 6962 §3.1），其余扩展原样保真。
extHex	public static String extHex(X509Certificate cert, String oid)	证书扩展值（OCTET STRING 解壳后的内容）→ hex；无此扩展返回空串。
treeFilePath	public static String treeFilePath(String baseDir, String logIdB64)	日志 Merkle 树文件路径（按 log_id 的 hex 命名，避免 base64 的 +/= 字符）。
loadTreeEntries	public static synchronized List<TreeEntry> loadTreeEntries(String baseDir, String logIdB64)	读取指定日志的 Merkle 树全部条目（JSON Lines，逐行解析）。
appendTreeEntry	public static synchronized void appendTreeEntry(String baseDir, String logIdB64, byte[] leafInput, byte[] extraData)	向指定日志的 Merkle 树追加一个条目（JSON Lines 追加一行，O(1)）。
buildX509LeafAndExtra	public static byte[][] buildX509LeafAndExtra(long ts, List<byte[]> chainDer)	构造 x509_entry 的 MerkleTreeLeaf（RFC 6962 §3.4）与 extra_data。 chainDer：完整证书链 DER（[0]=叶子证书，其余为颁发者链）。 返回 [leaf_input, extra_data]。
buildPrecertLeafAndExtra	public static byte[][] buildPrecertLeafAndExtra(long ts, byte[] issuerKeyHash, byte[] tbsNoPoison, List<byte[]> chainDer)	构造 precert_entry 的 MerkleTreeLeaf（RFC 6962 §3.4）与 extra_data。 chainDer：完整链 DER（[0]=预证书，其余为颁发者链）。 返回 [leaf_input, extra_data]。
leafHash	public static byte[] leafHash(byte[] leafInput) throws Exception	MerkleTreeLeaf 的叶子哈希 = SHA-256(0x00 || leaf_input)（RFC 6962 §3.4）。
mthRoot	public static byte[] mthRoot(List<byte[]> hashes) throws Exception	整棵树根哈希 MTH(D[0:n])；空树返回 32 字节全零。
auditPath	public static List<byte[]> auditPath(List<byte[]> hashes, int m) throws Exception	叶子索引 m 的 Merkle 审计路径（RFC 6962 §3.4.2，顺序：近叶 → 近根）。
consistencyProof	public static List<byte[]> consistencyProof(List<byte[]> hashes, int first, int second) throws Exception	两棵树头之间的一致性证明（RFC 6962 §3.4.3 CONSISTENCY_PROOF）。 hashes：当前树（size=second）的全部叶子哈希；first/second 为两个 tree_size（first<second）。
treeHeadSignature	public static byte[] treeHeadSignature(LogKey k, long ts, byte[] rootHash) throws Exception	TreeHeadSignature（RFC 6962 §3.5）：version(0) + signature_type=tree_hash(1) + timestamp(8) + root_hash(32)，用日志密钥签名。
buildSctInputX509	public static byte[] buildSctInputX509(long ts, byte[] leafCertDer)	RFC 6962 §3.2 SCT 签名输入（x509_entry 版：signed_entry = 叶子证书 DER）。

方法	签名	说明
signOneSct	public static SctOutcome signOneSct(String baseDir, LogKey k, int entryType, long ts, byte[] tbsNoPoison, byte[] issuerKeyHash, List<byte[]> chainDer) throws Exception	用单个日志密钥签发 SCT（RFC 6962 §4.1/§4.2 add-chain/add-pre-chain 语义），并把 MerkleTreeLeaf 追加到该日志的 Merkle 树（leaf 与 SCT 使用同一 timestamp）。 @param entryType 0=x509_entry（add-chain），1=precert_entry（add-pre-chain） @param tbsNoPoison
treeEntryTimestamp	public static long treeEntryTimestamp(byte[] leafInput)	从 MerkleTreeLeaf 解析 timestamp（version+leaf_type 各 1 字节后的 8 字节大端）。
b64ToHex	public static String b64ToHex(String b64)	base64 → hex。

MLog —— MLog.java

常量

常量	值	说明
TIME_FMT	DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm:ss")	时间格式：yyyy/MM/dd HH:mm:ss

方法一览（5 个）

方法	签名	说明
append	public static void append(String baseDir, String operation, String result)	追加一条普通日志（旧签名兼容：无命令字段，命令留空）。 @param baseDir MODPKICA 根目录 @param operation 操作名（命令 / 动作） @param result 结果 / 详情
append	public static void append(String baseDir, String operation, String command, String result)	追加一条普通日志。 @param baseDir MODPKICA 根目录 @param operation 操作名（命令 / 动作） @param command 输入的完整命令 / 请求数据（可为空串） @param result 输出结果 / 响应详情
appendErr	public static void appendErr(String baseDir, String operation, String command, String error)	追加一条错误日志（含报错详情）。 @param baseDir MODPKICA 根目录 @param operation 操作名（命令 / 动作） @param command 输入的完整命令 / 请求数据（可为空串） @param error 报错详情
write	private static synchronized void write(String baseDir, String operation, String command, String result, String error)	统一写入：时间 / 时间戳 / 操作 / 命令 / 结果 / 错误，条目间双换行。多线程安全（追加串行化）。
sanitize	private static String sanitize(String s)	清洗内容：换行转义为 \n，避免破坏条目结构。

OcspUtil —— OcspUtil.java

常量

常量	值	说明
OID_NONCE	"1.3.6.1.5.5.7.48.1.2"	Nonce 扩展 OID：1.3.6.1.5.5.7.48.1.2
OID_BASIC	OCSPObjectIdentifiers.id_pkix_ocsp_basic	id-pkix-ocsp-basic
GOOD	0	证书状态：正常
REVOKED	1	证书状态：已吊销
UNKNOWN	2	证书状态：未知

方法一览（7 个）

方法	签名	说明
parseRequest	public static OcspRequest parseRequest(byte[] der) throws Exception	解析 DER 编码的 OCSP 请求。

方法	签名	说明
createResponse	public static byte[] createResponse(String keyType, X509Certificate issuerCert, X509Certificate responderCert, ContentSigner signer, org.bouncycastle.crypto.params.ECPrivateKeyParameters sm2Priv, BigInteger serial, String hashAlgOid, int status, int reason, Date revokedAt, byte[] nonce) throws Exception	统一入口：按颁发者密钥类型生成 OCSP 响应。 @param keyType "RSA" / "ECC" / "SM2" @param issuerCert 签发该证书的 CA 证书 @param responderCert OCSP 响应器证书 @param signer ContentSigner（RSA/ECC 用） @param sm2Priv SM2 私钥参数（SM2 时
createResponseRsaEcc	private static byte[] createResponseRsaEcc(X509Certificate issuerCert, X509Certificate responderCert, ContentSigner signer, BigInteger serial, String hashAlgOid, int status, int reason, Date revokedAt, Date thisUpdate, Date nextUpdate, Date producedAt, byte[] nonce) throws Exception	============================ RSA / ECC ============================
createResponseSm2	private static byte[] createResponseSm2(X509Certificate issuerCert, X509Certificate responderCert, org.bouncycastle.crypto.params.ECPrivateKeyParameters sm2Priv, BigInteger serial, String hashAlgOid, int status, int reason, Date revokedAt, Date thisUpdate, Date nextUpdate, Date producedAt, byte[] nonce) throws Exception	============================ SM2 手动 ASN.1 ============================
matchesIssuerHash	public static boolean matchesIssuerHash(X509Certificate issuerCert, String hashAlgOid, byte[] reqNameHash, byte[] reqKeyHash)	校验请求 CertID 中的 issuerNameHash / issuerKeyHash 是否与颁发者证书一致 （P1-06：不同颁发者下相同序列号不得互相误命中）。 <p>口径与响应构造完全一致：issuerKeyHash 取 SPKI BIT STRING 裸公钥字节， issuerNameHash 取 issuer 证书 Subject 的规范 DER 编码。 @param issuerCert 颁发者证书 @param ha
hashDigestName	private static String hashDigestName(String oid)	把哈希 OID 映射为 JCA 摘要名，用于回算 issuerNameHash/issuerKeyHash。
unauthorizedRespons e	public static byte[] unauthorizedResponse()	未授权错误响应：固定 5 字节 {0x30,0x03,0x0A,0x01,0x06}。

PemIO —— PemIO.java

方法一览（12 个）

方法	签名	说明
readCert	public static X509Certificate readCert(String path) throws Exception	读取 PEM 证书文件为 X509Certificate。 @param path crt 文件路径 @return X509Certificate
readPrivateKey	public static AsymmetricKeyParameter readPrivateKey(String path) throws Exception	读取私钥 PEM，自动识别 RSA / ECC / SM2，返回 BC 的 AsymmetricKeyParameter。 <p>注意：BC 的 PEMParser 不识别国密自定义头 SM2 PRIVATE KEY（会直接抛 "unrecognised object"），因此这里手动解析 PEM 块： SM2 PRIVATE KEY / PRIVATE KEY → 内容即 PKCS#8 PrivateKeyInfo<
readPrivateKeyAsSign er	public static Object readPrivateKeyAsSigner(String path) throws Exception	读取私钥并直接返回可用于签名的对象（BC 私钥参数）。
writeCert	public static void writeCert(String path, X509Certificate cert) throws Exception	把 X509Certificate 写成 PEM（Type=CERTIFICATE）。 @param path 输出 crt 文件路径 @param cert 证书
writePrivateKey	public static void writePrivateKey(String path, Object key, String keyType) throws Exception	写私钥 PEM。 @param path 输出 key 文件路径 @param key 私钥（可为 AsymmetricKeyParameter 或 JCA PrivateKey） @param keyType "RSA" / "ECC" / "SM2"
detectKeyType	public static String detectKeyType(String pemPath) throws Exception	根据 PEM 头部 Type 判断密钥类型。 @param pemPath PEM 文件路径 @return "RSA" / "ECC" / "SM2"；无法识别返回 null
isPqcOid	public static boolean isPqcOid(String oid)	判断算法 OID 是否属于 PQC（后量子）算法族。 ML-DSA / SLH-DSA / ML-KEM / Falcon / XMSS 等（RFC 8708 / NIST PQC 标准 OID）。
isPqcAlgName	public static boolean isPqcAlgName(String alg)	判断算法名是否为 PQC（BC 返回的公钥/私钥算法名）。
lookupCurveBitLength	private static int lookupCurveBitLength(ASN1ObjectIdentifier oid)	按 OID 查曲线位宽，查不到默认 256。
readPemObject	private static PemObject readPemObject(String path) throws Exception	读取文件并解析为单个 PemObject。
writePemObject	private static void writePemObject(String path, PemObject po) throws Exception	把 PemObject 写出到文件（原子写：tmp → fsync → 备份 → move，避免断电/并发读到半截文件）。

方法	签名	说明
atomicWrite	public static void atomicWrite(Path out, byte[] bytes) throws Exception	原子写文件（对齐 CertsDB.rewriteAll：tmp → fsync → .bak → move），POSIX 上私钥收紧为 600。

SignTool —— SignTool.java

常量

常量	值	说明
OID_SPC_INDIRECT_DATA	"1.3.6.1.4.1.311.2.1.4"	spcIndirectDataContext：Authenticode 的 encapContentInfo.contentType
OID_SPC_PE_IMAGE_DATA	"1.3.6.1.4.1.311.2.1.15"	SpcPeImageData
OID_SPC_RFC3161_TS	"1.3.6.1.4.1.311.3.3.1"	Authenticode 时间戳属性（RFC3161 timestamp）
OID_SM3	"1.2.156.10197.1.401"	SM3 摘要算法 OID
OID_SM2	"1.2.156.10197.1.501"	SM2withSM3 签名算法 OID
OID_GOV_EXT	"1.2.156.112600.8.1"	政务体系私有扩展 OID（SESeal / 印章承载，GB/T 38540 extDatas / 35275 signedAttrs）
OID_TSA_TOKEN	"1.2.840.113549.1.9.16.2.14"	id-aa-timeStampToken（CAdES-T/PAdES-T 签名时间戳：SignerInfo unsignedAttrs）
SUBFILTER_GM_SEAL	"GM.sm2seal"	PDF 国密签章 SubFilter（GB/T 38540-2020：SES_Signature 结构）
SUBFILTER_GM_CMS	"GM.sm2cms.detached"	PDF 国密签章 SubFilter（GB/T 35275-2017：CMS SignedData + 私有 OID 印章）
SUBFILTER_ETSI_RFC3161	"ETSI.RFC3161"	PAdES 文档时间戳 SubFilter（RFC 3161 TimeStampToken 独立签名）。
EKU_CODE_SIGNING	"1.3.6.1.5.5.7.3.3"	代码签名相关 EKU（用于证书筛选）
EKU_KERNEL_MODE	"1.3.6.1.4.1.311.61.1.1"	代码签名相关 EKU（用于证书筛选）
EKU_DRIVER_VERIFY	"1.3.6.1.4.1.311.10.3.5"	代码签名相关 EKU（用于证书筛选）
EKU_SYS_COMPONENT	"1.3.6.1.4.1.311.10.3.6"	代码签名相关 EKU（用于证书筛选）
EKU_OEM_SYS	"1.3.6.1.4.1.311.10.3.7"	代码签名相关 EKU（用于证书筛选）
EKU_EMBED_SYS	"1.3.6.1.4.1.311.10.3.8"	代码签名相关 EKU（用于证书筛选）
EKU_DOC_MICROSOFT	"1.3.6.1.4.1.311.10.3.12"	代码签名相关 EKU（用于证书筛选）
EKU_DOC_GENERIC	"1.3.6.1.5.5.7.3.36"	内嵌系统组件 微软文档签名
imgW	bi.getWidth()	—
imgH	bi.getHeight()	—
is38540	sealType.contains("38540") || sealType.contains("0031")	—
is0031	sealType.contains("0031")	—
gmStd	is0031 ? "0031-2025" : is38540 ? "38540-2020" : "35275-2017"	—
isSeal	is38540 || is0031	—
fTsUrl	tsUrl	独立 SES_Signature 结构（GM.sm2seal）
fBase	base	独立 SES_Signature 结构（GM.sm2seal）

方法一览（69 个）

方法	签名	说明
isCodeSignCert	public static boolean isCodeSignCert(X509Certificate cert)	判定证书是否可用于代码签名（EKU 含标准或微软代码签名 OID）。
isDocSignCert	public static boolean isDocSignCert(X509Certificate cert)	判定证书是否可用于 PDF 文档签名（EKU 含微软文档签名或通用文档签名 OID 即可）。
ekusOf	public static Set<String> ekusOf(X509Certificate cert)	读取证书 EKU OID 集合（无 EKU 扩展返回空集合）。
keyBits	public static int keyBits(X509Certificate cert)	证书密钥位数（RSA modulus bit 数；ECC/SM2 曲线位宽）。

方法	签名	说明
loadCertBySerial	public static X509Certificate loadCertBySerial(String baseDir, String serial) throws Exception	按序列号从 CERTS.txt 台账定位并加载证书（R→ROOT、C→CA、E→CERT，兜底 WebPublic/CRT）。
loadKeyBySerial	public static PrivateKey loadKeyBySerial(String baseDir, String serial) throws Exception	按序列号加载私钥（R→ROOT、C→CA、E→KEY）。
loadChain	public static List<X509Certificate> loadChain(String baseDir, String serial) throws Exception	按序列号加载完整证书链（签名证书 + 按 CERTS.txt 颁发者关系逐级上级，直到自签根）。
certFilePath	private static String certFilePath(String baseDir, String serial)	—
digestName	public static String digestName(String keyType)	密钥类型 → 摘要名（"SHA256" / "SM3"）。
digestJca	public static String digestJca(String keyType)	密钥类型 → JCA 摘要算法名。
digestOid	public static String digestOid(String keyType)	密钥类型 → 摘要算法 OID。
sigAlgOid	public static String sigAlgOid(String keyType)	密钥类型 → 签名算法 OID（Authenticode SignerInfo.signatureAlgorithm）。
jcaSigAlg	public static String jcaSigAlg(String keyType)	密钥类型 → JCA 签名算法名。
digestJcaFromOid	private static String digestJcaFromOid(String oid)	摘要算法 OID → JCA 名（验证时从 CMS 读取）。
jcaSigFromOid	private static String jcaSigFromOid(String oid)	签名算法 OID → JCA 名（验证时使用）。
signExe	public static byte[] signExe(byte[] file, String keyType, X509Certificate cert, PrivateKey key, List<X509Certificate> chain, String baseDir) throws Exception	对 PE 文件（exe/dll/sys）做 Windows Authenticode 代码签名。 结构对齐 signtool 标准：手动 ASN.1（直接 SEQUENCE eContent、signedAttrs={contentType,messageDigest}、DigestInfo 带 NULL、SpcPeImageData 空 SpcLink），并自动请求时间戳嵌入。 @param file 原始 PE 字节 @p
signExe	public static byte[] signExe(byte[] file, String keyType, X509Certificate cert, PrivateKey key, List<X509Certificate> chain, String baseDir, String tsType, String tsUrl) throws Exception	对 PE 文件做 Authenticode 签名（可指定时间戳类型与地址）。 @param tsType RFC3161（标准 TimeStampToken，默认）/ AUTHENTICODE（微软老式 PKCS#7 时间戳） @param tsUrl 时间戳请求地址；null/空 = 默认本机 MAIN（RFC3161→/rfc3161，Authenticode→/timestamp）
authenticodeDigest	private static void authenticodeDigest(MessageDigest md, byte[] file, long checksumOff, long secDirOff, long certTableOff, long certTableSize)	Authenticode 哈希：跳过 checksum(4B)、Security Directory entry(8B)、证书表区域。
buildSpcIndirectData	private static byte[] buildSpcIndirectData(byte[] fileHash, String digestOid) throws Exception	构建 SpcIndirectDataContent（对齐 signtool 标准样本）： SpcPeImageData.file = [0] EXPLICIT → [2] EXPLICIT → SpcString(空 80 00)； SpcAttributeTypeAndOptionalValue.value = 直接 SEQUENCE（微软实测无 [0] 包装）； DigestInfo = 摘要算法(NULL) + 文件哈希。
buildExeCms	private static byte[] buildExeCms(byte[] spcDer, String keyType, X509Certificate cert, PrivateKey key, List<X509Certificate> chain, String baseDir, String tsType, String tsUrl) throws Exception	手动构造 Authenticode SignedData（对齐 signtool 标准，eContent 直接 SEQUENCE 无 OCTET STRING 包装）： SignedData v1, digestAlgorithms={摘要+NULL}, encapContentInfo={spcIndirectDataContext, [0] EXPLICIT SpcIndirectData}, certificates[0]={根在
verifyExe	public static String verifyExe(byte[] exe) throws Exception	验证 PE 文件代码签名：CMS 签名有效、Authenticode 文件哈希一致、证书链可建立。 eContent 为直接 SEQUENCE（无 OCTET STRING），故先手工提取内容字节，再以带 signedContent 的 CMSSignedData 构造验证（BC 才能按内容字节计算 messageDigest 比对）。
verifyChain	private static boolean verifyChain(StringBuilder sb, X509Certificate signerCert, Collection<X509CertificateHolder> certs, JcaX509CertificateConverter conv)	逐级验证签名证书 → 颁发者 → … → 自签根。
signPdf	public static byte[] signPdf(byte[] pdf, String keyType, X509Certificate cert, PrivateKey key, List<X509Certificate> chain, byte[] imageBytes) throws Exception	对 PDF 做标准 PKCS#7 detached 签名（adbe.pkcs7.detached）。 基于 PDFBox 3.0.4 外部签名两段式：先得到 ByteRange 待签数据，签名后 setSignature 写入。 @param imageBytes 可选签章图片（png/jpg/bmp/gif）；null 或空 = 不可见签名

方法	签名	说明
signPdf	public static byte[] signPdf(byte[] pdf, String keyType, X509Certificate cert, PrivateKey key, List<X509Certificate> chain, byte[] imageBytes, String sealType, String tsUrl) throws Exception	对 PDF 签名（可选签章类型与时间戳地址）。 @param sealType ADOBE（标准 PKCS#7 detached，默认）/ GM38540（GB/T 38540-2020 SES_Signature）/ GM35275（GB/T 35275-2017 CMS+私有OID印章）/ GM0031（GM/T 0031-2025 两层 SES_Signature） @param tsUrl PDF 时间戳地址（RFC316
signPdfTimestampOnly	public static byte[] signPdfTimestampOnly(byte[] pdf, String tsUrl) throws Exception	纯文档时间戳（无签名证书场景）：独立 DocTimeStamp 签名字段， Filter=Adobe.PPKLite、SubFilter=ETSI.RFC3161、Contents=RFC 3161 TimeStampToken（覆盖整个文档）。 适用于"不选证书、仅对文档盖时间戳"。 @param tsUrl RFC3161 时间戳地址；null/空 = 本机 MAIN /rfc3161
addDocTimeStamp	private static byte[] addDocTimeStamp(byte[] pdf, String tsUrl, String base) throws Exception	PAdES 文档时间戳（DocTimeStamp）：独立签名字段，Filter=Adobe.PPKLite、SubFilter=ETSI.RFC3161、Contents=RFC 3161 TimeStampToken（覆盖整个文档）。 仅在"无签名证书、纯文档时间戳"场景使用。 <p>同时嵌入 LTV（DSS：TSA 证书链 + 链上 OCSP 响应 + CRL），与时间戳同一次保存 （被 DocTimeStamp 覆盖，签名后零增
extractTsaChain	private static List<X509Certificate> extractTsaChain(byte[] tok) throws Exception	从 RFC3161 TimeStampToken 提取 TSA 证书链（叶子在前、根在后）。 token 内证书不全时返回单元素链（TSA 证书自身）。
serialHex	private static String serialHex(X509Certificate c)	证书序列号十六进制（小写，无 0x 前缀）。
signPdfAdobeFinal	private static byte[] signPdfAdobeFinal(byte[] pdf, String keyType, X509Certificate cert, PrivateKey key, List<X509Certificate> chain, byte[] imageBytes, String tsUrl, String base) throws Exception	Adobe 标准签名最终保存：数字签名 + DSS 同一次保存（数字签名是文档最后一次修改，签名后零增量）。 时间戳已按标准嵌入签名内部（CAdES-T unsignedAttrs 的 id-aa-timeStampToken）；DSS 的 Certs/OCSPs/CRLs 作为 PAdES 全局验证数据被签名覆盖（数字签名 ByteRange 无法在保存前预知，故省略 VRI 索引， 验证器按证书/响应内容匹配 DSS 数据）。 LT
buildAdobeSignature	private static PDSignature buildAdobeSignature(X509Certificate cert)	构建 Adobe 数字签名字段。
buildVisibleSignatureOptions	private static SignatureOptions buildVisibleSignatureOptions(PDDocument doc, byte[] imageBytes) throws Exception	构建可见签章选项（参考 java-lening 的 SignatureOptions 外观文档模式）： 独立外观 PDF 内放置带 Rect 与 AP 的签名 widget，PDFBox 会把 Rect/外观搬到 主文档第 1 页右下角，避免在 addSignature 之后改动文档破坏 ByteRange。
signPdfGm	private static byte[] signPdfGm(byte[] pdf, String sealType, String keyType, X509Certificate cert, PrivateKey key, List<X509Certificate> chain, byte[] imageBytes, String tsUrl, String base) throws Exception	—
sign	public byte[] sign(InputStream content) throws IOException	结构头 / 摘要 / 签名值 / 属性
buildGmSealInfo38540	private static byte[] buildGmSealInfo38540(X509Certificate cert, List<X509Certificate> chain, byte[] imageBytes, int w, int h) throws Exception	GB/T 38540-2020：构建 SES_SealInfo 印章信息（header + esID + property + picture + extDatas）。
buildGmSealInfo35275	private static byte[] buildGmSealInfo35275(X509Certificate cert, List<X509Certificate> chain, byte[] imageBytes, int w, int h) throws Exception	GB/T 35275-2017：构建 eSealInfo（sealId + UUID + certInfo + imgInfo + 扩展）。
buildGm38540Signature	private static byte[] buildGm38540Signature(byte[] docHash, X509Certificate cert, PrivateKey key, byte[] eSealInfo, byte[] sealSigned, String tsUrl, String base) throws Exception	GB/T 38540-2020 电子签章。时间戳支持（标准 7.1.6）：SES_Signature 可选第 5 字段 timeStamp（OCTET STRING）——对签名值的时间戳，符合 GB/T 20520（= RFC3161）， DER 编码存放；SM2/SM3 场景 messageImprint 用 SM3。 @param tsUrl RFC3161 时间戳地址；null = 不嵌时间戳；空串 = 本机 MAI
buildGm35275Signature	private static byte[] buildGm35275Signature(byte[] rangeData, byte[] docHash, X509Certificate cert, PrivateKey key, List<X509Certificate> chain, byte[] eSealInfo, byte[] sealSigned, byte[] certChainBytes, String tsUrl, String base) throws Exception	GB/T 35275-2017 CMS 签名。时间戳支持：标准 CMS SignerInfo 的 unsignedAttrs 可承载 CAdES-T 签名时间戳（id-aa-timeStampToken）——GB/T 35275 基于 CMS（ISO 8825），unsignedAttrs 为通用结构；messageImprint 用 SM3（SM2/SM3 证书固定 SM3）。 @param tsUrl RFC3161 时间戳地址；

方法	签名	说明
buildGmCertChainBytes	private static byte[] buildGmCertChainBytes(X509Certificate cert, List<X509Certificate> chain) throws Exception	证书链字节块：叶子在前、根在后顺序拼接（对齐参考项目 sm2sign‖ca）。
sm2Sign	private static byte[] sm2Sign(PrivateKey privateKey, byte[] data) throws Exception	SM2 签名（SM3withSM2，BC 实现含 ZA 计算，符合 GB/T 32918）。
buildCms	private static byte[] buildCms(byte[] content, String contentType, boolean encapsulate, String keyType, X509Certificate cert, PrivateKey key, List<X509Certificate> chain) throws Exception	生成 CMS SignedData（PDF detached 用 BC 生成器即可，标准 id-data 结构）。
buildCms	private static byte[] buildCms(byte[] content, String contentType, boolean encapsulate, String keyType, X509Certificate cert, PrivateKey key, List<X509Certificate> chain, String tsUrl) throws Exception	生成 CMS SignedData（PDF detached 用 BC 生成器即可，标准 id-data 结构）。 @param tsUrl PDF 时间戳地址（RFC3161）；null = 不嵌时间戳； 非 null（含空串）= 嵌入，空串 = 本机 MAIN /rfc3161。 嵌入方式：CAdES-T / PAdES-T 签名时间戳——token 放入 SignerInfo 的 unsignedAttrs
getContentType	public ASN1ObjectIdentifier getContentType()	BC 默认 signedAttrs：contentType / messageDigest / signingTime
write	public void write(OutputStream out) throws IOException	—
getContent	public Object getContent()	—
applyLtvPdf	public static byte[] applyLtvPdf(byte[] signedPdf, List<X509Certificate> chain, String baseDir)	127.0.0.1:WEBPORT/...）
collectLtvData	private static LtvData collectLtvData(List<X509Certificate> chain, String baseDir)	一次性收集 LTV 数据：证书链 DER + 链上各证书 OCSP 响应 + 链上 CA/根 CRL。失败返回 null。
installDss	private static void installDss(PDDocument doc, LtvData ltv, String vriKey) throws Exception	在文档中安装 DSS/VRI 对象（新增间接对象 + catalog /DSS 修订 + root 标记更新）。
newLtvStream	private static COSStream newLtvStream(byte[] data) throws IOException	创建承载 LTV 数据的流对象（字节写入，保存时由 COSObject 包装为间接对象）。
requestOcspResponse	private static byte[] requestOcspResponse(X509Certificate target, X509Certificate issuer, String baseDir)	RFC 6960 OCSP 请求（SHA-1 CertID + Nonce）→ 完整 OCSPResp DER；失败返回 null。
fetchIssuerCrl	private static byte[] fetchIssuerCrl(X509Certificate issuer, String baseDir)	下载颁发者 CRL（DER）。优先 CONFIG CRL URL 的 .der.crl 变体，回退原 URL 与本地文件。
configValue	private static String configValue(String baseDir, String key)	读取 CONFIG.txt 键值（缺省返回 def）。
configValue	private static String configValue(String baseDir, String key, String def)	读取 CONFIG.txt 键值（缺省返回 def）。
httpPost	private static byte[] httpPost(String url, String contentType, byte[] body)	HTTP POST（二进制），非 200 或异常返回 null。
httpGet	private static byte[] httpGet(String url)	HTTP GET，非 200 或异常返回 null。
isDer	private static boolean isDer(byte[] b)	判断字节是否为 DER 编码（可解析 ASN.1）。
toDerCrl	private static byte[] toDerCrl(byte[] pem)	PEM CRL（-----BEGIN X509 CRL-----）→ DER；非 PEM 原样返回。
verifyPdf	public static String verifyPdf(byte[] pdf) throws Exception	验证 PDF 签名：逐个签名字段（DocTimeStamp / 数字签名 / 国密），CMS 验签、ByteRange 摘要比对、证书链。
verifyDocTimeStamp	private static boolean verifyDocTimeStamp(StringBuilder sb, byte[] pdf, PDSignature sig)	验证 PAdES 文档时间戳（DocTimeStamp，SubFilter=ETSI.RFC3161）： Contents 为 RFC 3161 TimeStampToken；校验 messageImprint 与 ByteRange 摘要一致， 并用 token 内 TSA 证书验证签名。
verifyGmSeal38540	private static String verifyGmSeal38540(StringBuilder sb, byte[] contents, byte[] range) throws Exception	验证国密 SES_Signature（PDF Contents 原始结构，非 CMS）： 支持 GB/T 38540-2020（4 字段：toSign/cert/signatureAlgID/signature）与 GM/T 0031（2 字段：toSign/signature，证书与算法在 TBS_Sign 内部）： 1) 解析 SES_Signature；2) dataHash 与 SM3(ByteRange) 比对； 3) 用证书
shortDn	private static String shortDn(String dn)	—
printable	private static String printable(String s)	—
asn1ObjectTotalLength	private static int asn1ObjectTotalLength(byte[] d, int off)	解析 DER SEQUENCE 对象实际编码总长度（tag + 长度字段 + content），从指定偏移开始。
readU16	private static int readU16(byte[] b, int off)	解析 DER SEQUENCE 对象实际编码总长度（tag + 长度字段 + content），从指定偏移开始。
readU32	private static long readU32(byte[] b, int off)	跳过 tag(0x30)
writeU32	private static void writeU32(byte[] b, int off, int val)	—
writeU32	private static void writeU32(ByteArrayOutputStream out, int val)	—

方法	签名	说明
writeU16	private static void writeU16(ByteArrayOutputStream out, int val)	—
readAllBytes	private static byte[] readAllBytes(InputStream in) throws IOException	—
boundToSignature	private static boolean boundToSignature(TimeStampTokenInfo info, byte[] signature)	校验时间戳 messageImprint 是否绑定签名值（按 token 哈希算法自适应 SHA-256/SM3 等）。

Sm2Util —— Sm2Util.java

常量

常量	值	说明
OID_SM3_WITH_SM2	"1.2.156.10197.1.501"	SM3WithSM2 签名算法 OID：1.2.156.10197.1.501
OID_SM3	"1.2.156.10197.1.401"	SM3 摘要算法 OID：1.2.156.10197.1.401
OID_SM2_PUBLIC_KEY	"1.2.156.10197.1.301"	SM2 公钥算法 OID：1.2.156.10197.1.301
DEFAULT_UID	"1234567812345678"	默认用户 ID（与 GB/T 32918、Go tjfoc default_uid 一致）
CURVE_NAME	"sm2p256v1"	SM2 曲线名

方法一览（11 个）

方法	签名	说明
getSm2UidBytes	public static byte[] getSm2UidBytes()	返回默认用户 ID 的 UTF-8 字节（16 字节）。
sm3	public static byte[] sm3(byte[] data)	SM3 摘要。 @param data 原始数据 @return 32 字节摘要
computeZa	public static byte[] computeZa(ECPublicKeyParameters pub, byte[] uid)	计算 SM2 公钥的 ZA 值（供需要显式预计算的场景使用）。 @param pub 公钥参数 @param uid 用户 ID 字节，为空则用默认 ID @return ZA 摘要（32 字节）
addFieldElement	private static void addFieldElement(SM3Digest d, BigInteger v)	把大整数补齐为 32 字节后喂给摘要器。
sm2Sign	public static byte[] sm2Sign(ECPrivateKeyParameters priv, byte[] tbs)	SM2 签名（对 TBS 原文直接签名，输出 DER 编码的 r‖s）。 @param priv SM2 私钥参数 @param tbs 待签名的 TBS 原文 @return DER 编码的 (r, s) 签名
sm2Verify	public static boolean sm2Verify(ECPublicKeyParameters pub, byte[] tbs, byte[] sig)	SM2 验签。 @param pub SM2 公钥参数 @param tbs 原 TBS 原文 @param sig DER 编码签名 @return 验签是否通过
generateSm2KeyPair	public static org.bouncycastle.crypto.AsymmetricCipherKeyPair generateSm2KeyPair()	生成 SM2 密钥对（基于 sm2p256v1 曲线）。 @return AsymmetricCipherKeyPair（含 ECPrivateKeyParameters / ECPublicKeyParameters）
generateSm2JcaKeyPair	public static KeyPair generateSm2JcaKeyPair() throws Exception	用 JCA 方式生成 SM2 KeyPair（便于写入证书 / PEM）。
toEcPublicParams	public static ECPublicKeyParameters toEcPublicParams(java.security.PublicKey pub)	把 JCA 的 BCECPublicKey 转为 ECPublicKeyParameters。
toEcPrivateParams	public static ECPrivateKeyParameters toEcPrivateParams(java.security.PrivateKey priv)	把 JCA 的 BCECPrivateKey 转为 ECPrivateKeyParameters。
main	public static void main(String[] args) throws Exception	简易自测：生成密钥对 → 签名 → 验签，确认 SM2Signer + ParametersWithID 链路可用。

TimestampUtil —— TimestampUtil.java

常量

常量	值	说明
TSA_POLICY_OID	new ASN1ObjectIdentifier("1.3.6.1.4.1.4146.1.95")	本系统 TSA 策略 OID（不可用 2.23.140.1.3）
digest	new org.bouncycastle.crypto.digests.SM3Digest()	SM2 TSA 用的 SM3 摘要计算器（BC 无内置 JCA SM3 DigestCalculator）。
buffer	new ByteArrayOutputStream()	SM2 TSA 用的 SM3 摘要计算器（BC 无内置 JCA SM3 DigestCalculator）。

方法一览（8 个）

方法	签名	说明
parseRfc3161Request	public static Rfc3161Request parseRfc3161Request(byte[] der) throws Exception	解析 DER 编码的 RFC3161 请求。
createRfc3161Response	public static byte[] createRfc3161Response(byte[] requestDer, X509Certificate tsaCert, List<X509Certificate> chain, ContentSigner signer, DigestCalculator digestCalc) throws Exception	生成 RFC3161 时间戳响应（DER）。 @param requestDer 原始请求 DER @param tsaCert TSA 证书 @param chain TSA 证书链（可为空） @param signer ContentSigner（SM2 时用 SM2ContentSigner） @param digestCalc 与请求 messageImprint 一致的摘要计算器（S
createRfc3161ResponseStandard	public static byte[] createRfc3161ResponseStandard(byte[] requestDer, X509Certificate tsaCert, List<X509Certificate> chain, PrivateKey tsaKey, String signatureAlgJca, String digestAlgJca) throws Exception	标准（RSA/ECC）RFC3161 响应便捷方法。
getAlgorithmIdentifier	public AlgorithmIdentifier getAlgorithmIdentifier()	SM2 TSA 用的 SM3 摘要计算器（BC 无内置 JCA SM3 DigestCalculator）。
getOutputStream	public OutputStream getOutputStream()	—
getDigest	public byte[] getDigest()	—
createAuthenticodeTimestamp	public static byte[] createAuthenticodeTimestamp(byte[] content, X509Certificate cert, PrivateKey key, int digitype) throws Exception	—
isEcc	private static boolean isEcc(PrivateKey key)	判断私钥是否为 ECC。

WebAdmin —— WebAdmin.java

常量

常量	值	说明
DEFAULT_PORT	8080	默认端口 8080。
DEFAULT_BIND	"127.0.0.1"	默认绑定地址：仅本机回环（管理界面不暴露公网）。
TOTAL_CALLS	new AtomicLong()	总 API 调用次数。
ENDPOINT_CALLS	new ConcurrentHashMap<>()	各端点调用计数。
WRITE_PATHS	java.util.Set.of("/api/config", "/api/init", "/api/makeroot", "/api/signcert", "/api/revoke", "/api/restore", "/api/rek	写 PKI 数据的管理 API（并发时串行执行，避免文件竞争）。 注意：凡改动 CERTS.txt / myCERTS.txt / CONFIG.txt / modpkica_ct_list.json / log_list.json / CRL / OCSP / TIMSTAMP / 证书与私钥文件 的接口都必须在此登记，否则并发时与其它写操作发生文件竞争（tmp→move 互相覆盖）。
WRITE_LOCK	new Object()	写操作全局锁：保证 PKI 文件（证书/私钥/CRL/CONFIG/日志）串行修改。
OID_EKU_TSA	"1.3.6.1.5.5.7.3.8"	EKU OID：TSA / OCSP 专用。
OID_EKU_OCSP	"1.3.6.1.5.5.7.3.9"	EKU OID：TSA / OCSP 专用。
PAGE_P1	""" <!DOCTYPE html> <html lang="zh-CN"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width,	从解析出的参数取字符串值。
PAGE_P1B	""" <!-- 初始化环境 --> 初始化环境（Admin init） 注意：	—
PAGE_P2	""" <script> // ===== 工具 ===== window.addEventListener('error', function(e){ var d=document.getElementById('jsErr')	—
PAGE_P2B	""" const walk = nodes => (nodes || []).forEach(n => { issuerMap[n.serial] = n	签发证书的颁发者下拉：CERTS.txt 中 R/C 类型序列号（悬停显示证书详情） /api/chain 返回嵌套树，展平为 serial → 节点
PAGE_P3	""" <script> // ===== 调试专区：模拟 OCSP / 时间戳请求 ===== async function sendDebugOcsp() { const out = $('dbgOcspOut')	—

常量	值	说明
PAGE_VIEWER	""" <script> /* ========== ========== ==========

方法	签名	说明
parseLocalDateTime	private static Date parseLocalDateTime(String s) throws Exception	解析前端 datetime-local 的本地时间（yyyy-MM-ddTHH:mm[:ss]），转 UTC Date。
signAndVerify	private static boolean signAndVerify(KeyPair kp, String jcaAlg, byte[] data) throws Exception	用 JCA Signature 对随机数据做签名并验证。
saveRekeyFiles	private static String saveRekeyFiles(String serial, CertsDB.CertEntry entry, byte[] certPem, KeyPair kp, String keyType) throws Exception	按证书身份定位保存位置并替换旧文件： OCSP 证书（OCSP/{颁发者ID}_ocsp.crt 序列号匹配）→ OCSP 目录； TSA 证书（TIMSTAMP 目录内证书序列号匹配）→ TIMSTAMP 目录；否则按登记类型 R→ROOT、C→CA、E→CERT/KEY。始终同步 WebPublic/CRT。
certSerialOf	private static String certSerialOf(Path certPath) throws Exception	读取证书文件的序列号（hex 小写）。
relPath	private static String relPath(Path p)	相对 baseDir 的路径显示。
statusJson	private static Map<String, Object> statusJson()	MAIN 服务状态 + WebAdmin 自身统计。
certsJson	private static Map<String, Object> certsJson() throws Exception	证书库列表。
myRecordPath	private static Path myRecordPath()	我的证书：PKI/MY 导入证书库（myCERTS.txt 台账 + {序列号}.crt/.key） ========== ========== ==========
myListJson	private static Map<String, Object> myListJson()	/api/my/list：读取 PKI/MY/myCERTS.txt 台账。
myHasSerial	private static boolean myHasSerial(String serial)	myCERTS.txt 是否已登记该序列号。
verifyTarget	private static String verifyTarget(String target)	验证目标回退：MY 台账中的序列号 → PKI/MY/序列号.crt 绝对路径，其余原样（CERTS.txt 行为不变）。
mySerialOf	private static String mySerialOf(java.security.cert.X509Certificate c)	证书序列号规范化：小写 hex，奇数补 0（与 CertVerify 显示一致）。
myAlgoOf	private static String myAlgoOf(java.security.cert.X509Certificate c)	证书算法：RSA / ECC / SM2 / PQC（ML-DSA/SLH-DSA/ML-KEM 等）。
myTypeOf	private static String myTypeOf(java.security.cert.X509Certificate c)	证书类型：自签名 CA → R；CA → C；终端 → E。
fmtMyTime	private static String fmtMyTime(java.util.Date d)	证书类型：自签名 CA → R；CA → C；终端 → E。
myKeyMatches	private static boolean myKeyMatches(java.security.cert.X509Certificate cert, PrivateKey priv)	校验证书与私钥匹配：私钥签名 → 证书公钥验签（SM2 走 Sm2Util，同一 ZA 上下文）。
myImportAction	private static void myImportAction(HttpExchange ex)	/api/my/import：导入 pfx/p12/jks（需密码）或 cer/crt/pem（含私钥可选）。
loadKeyStoreEntries	private static java.util.List<Object[]> loadKeyStoreEntries(byte[] data, String ksType, String pwd) throws Exception	读取 JKS / PKCS12 的全部私钥条目（私钥 + 证书链叶子）。 PKCS12 用 BC 直接加载；JKS 是 Sun 专有容器且私钥在 getKey() 时才解密，因此必须把「提 BC 为首位 → 加载 → 读取全部条目」放在同一作用域内完成，否则恢复位置后 Sun 的 EC KeyFactory 仍不认识 SM2 曲线（1.2.156.10197.1.301）。
collectKeyStoreEntries	private static void collectKeyStoreEntries(java.security.KeyStore ks, String pwd, java.util.List<Object[]> out, java.util.List<String> errs)	收集密钥库全部私钥条目（私钥 + 证书链叶子），单条目异常跳过并把失败原因记入 errs。
importKeyStore	private static Map<String, Object> importKeyStore(byte[] data, String fileName, String password) throws Exception	导入 JKS/PKCS12：导出全部私钥条目（公私钥不匹配的条目跳过），返回首条 + count/all。
persistMyCert	private static Map<String, Object> persistMyCert(java.security.cert.X509Certificate cert, PrivateKey priv) throws Exception	落盘 PKI/MY/{serial}.crt|.key 并登记台账（重复序列号先移除旧记录）。
myAppendRecord	private static void myAppendRecord(String rec) throws Exception	追加台账记录：文件不存在或为空时先写模板头。
myDelete	private static String myDelete(String serial) throws Exception	/api/my/delete：删除 PKI/MY/序列号.crt/.key，并从台账移除记录。
myRemoveRecord	private static void myRemoveRecord(String serial) throws Exception	—
certsDelete	private static String certsDelete(String serial) throws Exception	/api/certs/delete：将 CERTS.txt 中指定序列号行加 # 注释（忽略该行；生成 CRL/OCSP 时 CertsDB 自动跳过），不删除任何文件。
cacertsJson	private static Map<String, Object> cacertsJson()	/api/certs/cacerts：列出 JAVA 环境 cacerts 中的可信证书（结构同 /api/certs，只读）。
fmtCacertsTime	private static String fmtCacertsTime(java.util.Date d)	cacerts 列表时间格式：yyyy/MM/dd HH:mm:ss（本地时区）。

方法	签名	说明
loadCacertsCerts	private static java.util.List<java.security.cert.X509Certificate> loadCacertsCerts()	读取 JAVA 环境 cacerts（JKS/PKCS12，默认密码 changeit）中的全部证书。
certSerialHex	private static String certSerialHex(java.security.cert.X509Certificate c)	证书序列号 -> 小写 hex（补前导零）。
certAlgoDesc	private static String certAlgoDesc(java.security.cert.X509Certificate c)	证书公钥算法描述：RSA 2048 / EC 256 / DSA 1024 等。
findCacertsCertBySerial	private static byte[] findCacertsCertBySerial(String serial)	按序列号在 cacerts 中查找证书，返回 PEM 字节；未找到返回 null。
issuersJson	private static Map<String, Object> issuersJson() throws Exception	可作颁发者的序列号（CERTS.txt 中类型 R 或 C），供签发证书下拉选择。
logsJson	private static Map<String, Object> logsJson(HttpExchange ex)	日志接口：支持 lines=N（尾部行数）与 hours=N（最近 N 小时，0=全部）。
configJson	private static Map<String, Object> configJson()	系统配置接口（GET）：读取 PKI/CONFIG.txt 的 6 个已知参数。 文件不存在时返回默认模板值（exists=false）。
saveConfig	private static void saveConfig(HttpExchange ex) throws Exception	系统配置接口（POST）：参数化保存 CONFIG.txt（保留注释与未知键，缺失键自动补齐）。
entryMillis	private static long entryMillis(String entry)	从日志条目文本提取毫秒时间戳：优先「时间戳:」行，旧格式按首行时间解析。
chainJson	private static Map<String, Object> chainJson() throws Exception	从 CERTS.txt 构建证书链树： R（自签根）为顶层 → C（中间CA）挂其下 → E（终端证书）挂最下。 每个节点附带：序列号、DN、密钥对类型、哈希算法、OCSP 状态、CRL 状态。
buildChainNode	private static Map<String, Object> buildChainNode(CertsDB.CertEntry e)	构造单个链节点（含证书文件信息加载）。
scanSerialInDir	private static String scanSerialInDir(String dir, String serial)	扫描目录下全部 .crt，返回证书序列号与目标一致的相对路径。
hashFromCert	private static String hashFromCert(java.security.cert.X509Certificate c)	从证书签名算法推导哈希算法名（sha1/sha256/sha384/sha512/sm3）。
keyBits	private static int keyBits(java.security.cert.X509Certificate c)	从证书公钥推导密钥长度（位）：RSA=模数位长，ECC/SM2=曲线域大小。
certKind	private static String certKind(java.security.cert.X509Certificate c)	识别证书子类型（终端证书方括号内容，只能显示一种）： TSA/OCSP/文档签名靠 EKU；SSL(DV/IV/OV/EV)、代码签名(普通/EV)靠证书策略 OID；TSA 证书另有策略标记 OID 2.23.140.1.4.2。
policyOidSet	private static Set<String> policyOidSet(java.security.cert.X509Certificate c)	读取证书策略扩展（2.5.29.32）中的全部策略 OID。
certJson	private static Map<String, Object> certJson(HttpExchange ex) throws Exception	/api/certjson?serial=xxx：返回 X509JsonConverter 风格的证书 JSON 文本。
certChainJson	private static Map<String, Object> certChainJson(HttpExchange ex) throws Exception	/api/certchainjson?serial=xxx：按颁发者链递归返回每层证书 JSON（[终端, ..., 根]）。
certDownload	private static void certDownload(HttpExchange ex) throws Exception	/api/cert/download?serial=..&kind=..&password=..&src=certs|cacerts|my —— 下载文件（附件）。
certExport	private static void certExport(HttpExchange ex) throws Exception	/api/cert/export?serial=..&kind=..&src=.. —— 导出文本（剪贴板）。
qp	private static String qp(String q, String name)	query 参数提取（URL 解码，非法编码返回 null）。
normSerial	private static String normSerial(String serial)	序列号规范化：去 0x 前缀 + 小写 + [0-9a-f] 白名单校验。 返回 null 表示缺失/非法（调用方应返回 400）。所有拼接文件路径的 serial 必须经过本校验， 防止路径穿越读取任意 .crt/.key。
loadCertBySource	private static java.security.cert.X509Certificate loadCertBySource(String serial, String src) throws Exception	按序列号加载证书（src=certs|cacerts|my）。序列号经 normSerial 白名单校验。
buildCertChain	private static java.util.List<java.security.cert.X509Certificate> buildCertChain(String serial, String src) throws Exception	构建证书链（自身 → 颁发者 → … → 根）。
findIssuerByDn	private static String findIssuerByDn(java.security.cert.X509Certificate child, String src)	按颁发者 DN 在 ROOT/CA/MY/CERT/WebPublic 目录及 cacerts 中寻找 subject 匹配的上级证书。
findKeyFile	private static String findKeyFile(String serial, String src)	查找私钥文件路径（相对 baseDir；cacerts 无私钥返回 null）。序列号经白名单校验。
p7bBytes	private static byte[] p7bBytes(java.util.List<java.security.cert.X509Certificate> chain) throws Exception	PKCS#7 certificates-only（.p7b）。

方法	签名	说明
keyStoreBytes	private static byte[] keyStoreBytes(java.util.List<java.security.cert.X509Certificate> chain, java.security.PrivateKey key, String password, String type) throws Exception	PKCS12 / JKS 密钥库（证书链 + 私钥 + 密码）。
readJcaPrivateKey	private static java.security.PrivateKey readJcaPrivateKey(String keyPath) throws Exception	读取私钥文件为 JCA PrivateKey（支持 PKCS#1 / SEC1 / PKCS#8 / OpenSSH 等 PEM 类型）。
pemToDer	private static byte[] pemToDer(byte[] pemBytes)	PEM 字节 → DER 字节（已是 DER 时原样返回）。字符过滤方式，兼容任何 PEM 变体。
sendBytes	private static void sendBytes(HttpExchange ex, int code, String contentType, String fileName, byte[] data) throws IOException	发送二进制文件（附件下载）。
readConfigWebPort	private static String readConfigWebPort()	读取 CONFIG.txt 的 WEBPORT（供状态展示）。
countLogLines	private static long countLogLines()	logs.log 行数（缓冲读 + 30 秒缓存，避免每次状态请求全文件扫描）。
tailFile	private static String tailFile(String relPath, int lines)	读文件尾部最多 lines 行（仅定位换行起点，整段一次性 UTF-8 解码，避免分块解码截断多字节中文）。
readAll	private static byte[] readAll(InputStream in, long maxBytes) throws IOException	读取请求体全部字节（读中限流：超过 maxBytes 立即抛 IOException，避免大 body 先占满内存）。
readAll	private static byte[] readAll(InputStream in) throws IOException	读取请求体全部字节（默认读中上限 8MB，超过即中止）。
readMultipart	private static Map<String, byte[]> readMultipart(HttpExchange ex) throws Exception	解析 multipart/form-data 请求体，返回 字段名 → 字节。
indexOfBytes	private static int indexOfBytes(byte[] hay, byte[] needle, int from)	字节数组查找。
signAction	private static void signAction(HttpExchange ex, boolean pdf)	签名动作（pdf=true 时走 PDF 签名，否则 EXE Authenticode 签名）。
verifyAction	private static void verifyAction(HttpExchange ex, boolean pdf)	验证动作（pdf=true 走 PDF 验证，否则 EXE Authenticode 验证）。
verifyCertFileAction	private static void verifyCertFileAction(HttpExchange ex)	/api/verify/file：上传本地 cer/crt/pem 证书文件并执行完整验证（CertVerify）。
signCertsJson	private static Map<String, Object> signCertsJson(HttpExchange ex)	可签名证书列表：kind=exe 筛代码签名 EKU；kind=pdf 筛文档签名/代码签名 EKU。
ctlogListJson	private static Map<String, Object> ctlogListJson()	/api/ctlog/list：密钥库列表（不含私钥）。
ctlogLogsJson	private static Map<String, Object> ctlogLogsJson()	/api/ctlog/logs：ctlogs.txt 全部行（只读）。
ctlogPublicJson	private static Map<String, Object> ctlogPublicJson()	/api/ctlog/public：可信公网 CT 日志公钥列表（log_list.json，只读）。
ctlogPublicFetch	private static String ctlogPublicFetch() throws Exception	/api/ctlog/publicFetch：从 gstatic 拉取最新 log_list.json 并保存。
ctlogGen	private static String ctlogGen(String desc, String type) throws Exception	/api/ctlog/gen：生成日志密钥对（type=ECDSA /RSA/SM2，缺省 ECDSA）并写入密钥库。
ctlogDelete	private static String ctlogDelete(String logId)	/api/ctlog/delete：按 log_id 删除密钥对。
ctlogSetState	private static String ctlogSetState(String logId, String state)	/api/ctlog/enable、/api/ctlog/pause：修改密钥状态。
templatePath	private static Path templatePath()	目录名按用户指定拼写 "tamplate"；模板供"签发证书"一键回填与批量签署 ========== ========== ==========
fmtTemplateNow	private static String fmtTemplateNow()	目录名按用户指定拼写 "tamplate"；模板供"签发证书"一键回填与批量签署 ========== ========== ==========
templateLoad	private static List<Map<String, Object>> templateLoad()	读取全部模板（文件不存在/损坏时返回空数组）。
templateWrite	private static void templateWrite(List<Map<String, Object>> list) throws Exception	原子写模板文件（先写 tmp 再 move，避免半截 JSON）。
templateListJson	private static Map<String, Object> templateListJson()	/api/template/list：模板列表。
parseJsonArray	private static List<Map<String, Object>> parseJsonArray(String json)	解析 JSON 对象数组（对象为扁平 key/value，兼容 readJsonString 转义）。
templateSave	private static String templateSave(Map<String, Object> m) throws Exception	/api/template/save：新增（无 id）或更新（带 id）模板。
templateDelete	private static String templateDelete(Map<String, Object> m) throws Exception	/api/template/delete：按 id 删除模板。
templateSign	private static String templateSign(Map<String, Object> m) throws Exception	/api/template/sign：按模板参数 + 选中的 R/C 颁发者签发证书（复用 MakeCert 全流程，日志由 runAction/doSignCert 记录）。

方法	签名	说明
sendJson	private static void sendJson(HttpExchange ex, int code, Map<String, Object> map) throws IOException	—
appendJsonValue	private static void appendJsonValue(StringBuilder sb, Object v)	JSON 值序列化（支持 String/Number/Boolean/Map/List/null）。
sendText	private static void sendText(HttpExchange ex, int code, String contentType, String text) throws IOException	发送文本响应。
parseJsonObject	private static Map<String, Object> parseJsonObject(String json)	解析 JSON 对象为 Map（只处理扁平字符串值）。
readJsonString	private static Object[] readJsonString(String s, int i)	读取 JSON 字符串字面量（i 指向开引号后一字符），返回 [解码值, 闭引号后一位置]。 支持 \\n \\r \\t \\b \\f \\/ \\" \\\\ 与 uXXXX 十六进制转义（未闭合时返回剩余内容）。
str	private static String str(Map<String, Object> m, String key)	从解析出的参数取字符串值。

X509JsonConverter —— X509JsonConverter.java

常量

常量	值	说明
VERSION	"1.0.0"	X.509 PEM 证书 <-> JSON 结构文本 双向无损转换器 设计原则： 1. JSON 纯可视化数据，不含任何 DER/hex 编码的扩展值字段 2. DN(issuer/subject) 以 OID 为主键，每个属性含 name/value/encoding 3. v3 扩展以 OID 为主键，每个扩展含 critical/name 及扩展特有字段 4. 字节级无损：通过保留字符串编码类型、时间类型、BIT STRING u
ATTR_NAMES	new HashMap<>()	BigInteger -> 偶数长度小写 hex（补前导零），与 ASN.1 INTEGER 原始字节一致
EXT_NAMES	new HashMap<>()	BigInteger -> 偶数长度小写 hex（补前导零），与 ASN.1 INTEGER 原始字节一致
SIG_ALG_NAMES	new HashMap<>()	BigInteger -> 偶数长度小写 hex（补前导零），与 ASN.1 INTEGER 原始字节一致
KEY_ALG_NAMES	new HashMap<>()	BigInteger -> 偶数长度小写 hex（补前导零），与 ASN.1 INTEGER 原始字节一致
EKU_NAMES	new HashMap<>()	BigInteger -> 偶数长度小写 hex（补前导零），与 ASN.1 INTEGER 原始字节一致
AIA_METHODS	new HashMap<>()	========== OID 名称映射（仅用于可读性） ==========
GN_TYPES	new HashMap<>()	========== OID 名称映射（仅用于可读性） ==========
HASH_ALGS	new HashMap<>()	========== SCT List (RFC 6962) 解析 ==========
SIG_ALGS	new HashMap<>()	========== SCT List (RFC 6962) 解析 ==========
HASH_ALG_CODES	new HashMap<>()	========== SCT List (RFC 6962) 解析 ==========
SIG_ALG_CODES	new HashMap<>()	========== SCT List (RFC 6962) 解析 ==========
REASON_NAMES	{"unused","keyCompromise","cACompromise","affiliationChanged", "superseded","cessationOfOperation","certificateHold","pr	========== ReasonFlags 位列表 <-> 掩码 ==========
REASON_BITS	{ReasonFlags.unused, ReasonFlags.keyCompromise, ReasonFlags.cACompromise, ReasonFlags.affiliationChanged, ReasonFlags.su	========== ReasonFlags 位列表 <-> 掩码 ==========

方法一览（20 个）

方法	签名	说明
main	public static void main(String[] args) throws Exception	========== 主入口 ==========
isHelpArg	private static boolean isHelpArg(String arg)	—
printUsage	private static void printUsage()	—
certToJson	public static String certToJson(byte[] pemBytes) throws Exception	========== PEM -> JSON ==========
jsonToCertPem	public static byte[] jsonToCertPem(String json) throws Exception	========== JSON -> PEM（从结构化数据重建 ASN.1） ==========
rekeyJsonToPemByIssuer	public static byte[] rekeyJsonToPemByIssuer(String json, KeyPair newKp, PrivateKey issuerKey, String issuerKeyType) throws Exception	—
renewJsonToPemByIssuer	public static byte[] renewJsonToPemByIssuer(String json, Date notBefore, Date notAfter, PrivateKey issuerKey, String issuerKeyType) throws Exception	续签证书（CA 重签版，WebAdmin 证书库「续签」使用）： 保持原公钥与原扩展，仅将 validity 的 notBefore / notAfter 更新为新时间， 重建 TBSCertificate → 用颁发者（CA/根）私钥签名 → 写回 signatureValue → 组装 Certificate → 输出 PEM。 @param json 证书 JSON（X509JsonConvert
buildAndSignCert	private static byte[] buildAndSignCert(Map<String, Object> root, PrivateKey issuerKey, String issuerKeyType) throws Exception	共用：重建 TBSCertificate → 颁发者私钥签名 → 写回 signatureValue → 组装 Certificate → PEM。
generateKeyPairFromJson	public static KeyPair generateKeyPairFromJson(String json) throws Exception	从证书 JSON 生成同类型新密钥对（供 WebAdmin 密钥更换调用）。 算法识别自 subjectPublicKeyInfo.algorithm.algorithm 的 OID。
keyTypeFromJson	public static String keyTypeFromJson(String json) throws Exception	从证书 JSON 识别密钥算法类型名：RSA / ECC / SM2（供 WebAdmin 密钥更换调用）。 注意：SM2 证书的 SPKI 算法 OID 是 ecPublicKey(1.2.840.10045.2.1)， SM2 曲线在 algorithm.parameters（hex 编码的 OID 1.2.156.10197.1.301）里， 必须按曲线判定，否则会把 SM2 误判为普通 ECC。
curveOidFromParams	private static String curveOidFromParams(Object params)	从 SPKI algorithm.parameters 提取曲线 OID（parameters 为 DER OID 的 hex 或名称）。
doRekey	private static byte[] doRekey(Map<String, Object> root, KeyPair newKp) throws Exception	rekey 内部共用逻辑：替换公钥 → 构建 TBS → 签名 → 组装证书
value	private Object value()	========== JSON 解析器 ==========
object	private Map<String, Object> object()	—
array	private List<Object> array()	:
string	private String string()	—
bool	private Boolean bool()	—
nul	private void nul()	—
number	private Number number()	—
skipWs	private void skipWs()	—

第五章 附录

5.1 构建脚本用法（build.cmd / build.sh）

```
build.cmd           # 无参数 = 全量编译（javac -Xlint:all，零警告基线）
build.cmd 1         # 编译成功后自动运行 MAIN（网络服务 80 + 管理面板 8080）
build.cmd 2         # 编译成功后自动运行 WebAdmin（仅管理面板）
build.cmd 3         # Admin 命令行入口
build.cmd 4         # MakeRoot 示例（"CN=Root,O=Org,C=CN" 2048 sha256 RSA）
build.cmd 5         # MakeCert 示例（"CN=www.test.com" ...）
build.cmd 6         # RootGetCrl 生成吊销列表
build.cmd 7         # CertVerify 证书验证
build.cmd 8         # Auto revoke 吊销示例
build.cmd 9         # 单元测试（debug_test，162 passed / 0 failed）
build.sh 等价（Linux/macOS，UTF-8 环境）。
```

5.2 全局配置参数（PKI/CONFIG.txt）

```
WEBPORT=80                    # 网络服务监听端口
OCSP=http://localhost:80/OCSP  # OCSP 响应中 AIA 的 OCSP URL
CRL=http://localhost:80/CRL/{CID}.crl     # CRL 分发点模板（{CID}=证书序列号）
CRT=http://localhost:80/CRT/{CID}.crt     # 证书发布 URL 模板
CPS=https://gitee.com/wxgshuju/modpkica  # 证书策略 CPS 链接
CPS_Nonice=http://example.com/nonice      # 用户通告（支持 \n 换行）
suport=8080                   # Web 管理面板端口（缺省 8080，缺失自动创建）
```

5.3 日志格式（PKI/logs.log）

每次操作写：当前时间、时间戳、执行的操作、输入的完整命令（Web
操作含提交的全部数据）、输出结果、报错详情，两条日志之间以两个换行符分隔；只追加不覆盖。